

FoodBridge - A Food Rescue Web Application Built Using Django and React.

Internship report submitted to

Dr. Babasaheb Ambedkar Technological University, Maharashtra

in partial fulfilment of the requirements for the award of

the degree

Bachelor of Technology

In

Computer Engineering

By

Sanket S. Prajapati

PRN No.: 23046491245506

under the guidance of

Dr. V. R. Palekar

Department of Computer Engineering



Department of Computer Engineering

Bajaj Institute of Technology

Wardha – 442 001 (India)

2025–26

BAJAJ INSTITUTE OF TECHNOLOGY, WARDHA
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE



This is to certify that the Internship report titled

**FoodBridge – A Food Rescue Web Application Built Using Django
and React.**

has been successfully completed by

Sanket S. Prajapati

PRN No.: 23046491245506

*in partial fulfilment of the requirements for the award of the degree in Bachelor of
Technology, Computer Engineering.*



Ms. Prajakta V. Murmure

Industry Manager
Kiran Academy, Pune

Dr. V. R. Palekar

Guide
Department of Computer Engineering
Bajaj Institute of Technology, Wardha

Prof. Sheetal Kale

Head of the Department
Department of Computer Engineering
Bajaj Institute of Technology, Wardha

Dr. N. M. Kanhe

Principal
Bajaj Institute of Technology, Wardha

Date: May 29, 2026

Place: Wardha

Serial No: JBK-INT-COMP-202604-000305

Date: 27-Apr-2026

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that Mr./Ms. Sanket Prajapati has successfully completed six months of Industrial Training with Internship at Kiran Academy, starting from the enrolment date.

During the training period, he/she worked as a Trainee Developer and was involved in the development of a Industrial Project under the guidance of Kiran Academy's Expert Technical Team.

Throughout the training tenure, he/she demonstrated strong technical skills, sincerity, dedication, and a strong learning attitude. We wish him/her all the best for future endeavours.

Technology: Python Full Stack

Authorized Signatory

Kiran Academy

Note: This document is auto-generated and does not carry a physical stamp or signature. To verify its authenticity, please email operations@thekiranacademy.com with the Serial Number mentioned above.

DECLARATION

I, hereby declare that the internship report titled "FoodBridge – A Food Rescue Web Application Built Using Django and React" submitted by me to the Bajaj Institute of Technology, Wardha, in partial fulfilment of the requirement for the award of the Degree of B.Tech. in Computer Engineering discipline is a record of bona fide project work carried out by me under the guidance of Dr. V. R. Palekar, Department of Computer Engineering, Bajaj Institute of Technology, Wardha, and Ms. Prajakta V Murmure, Industry Mentor, Kiran Academy, Pune.

I further declare that this submission by the undersigned represents my original work and I have quoted all references wherever others' ideas or words have been included. I understand that any violation of the above will attract disciplinary action.

I further declare that the work reported in this project report has not been submitted either in part or in full for the award of any other degree in any other Institute or University.

Sr. No.	Name of the Student	PRN Number	Signature
1	Sanket Prajapati	23046491245506	

Date:

Place: Wardha

ACKNOWLEDGEMENT

I am deeply grateful to Ms. Prajakta V. Murmure, Industry Mentor at Kiran Academy, Pune, for her invaluable guidance, constant encouragement, and unwavering support throughout the duration of this internship. Her practical insights into software development workflows and her constructive feedback at every milestone were instrumental in shaping the FoodBridge project into a fully functional, fully functional and deployable web application.

I express my sincere gratitude to Dr. V. R. Palekar, Faculty Guide, Department of Computer Engineering, Bajaj Institute of Technology, Wardha, for his academic guidance, technical suggestions, and continuous motivation throughout the completion of this report.

His expertise and availability for discussions were of immense value.

I am indebted to Prof. Sheetal Kale, Head of the Department of Computer Engineering, Bajaj Institute of Technology, Wardha, for providing a stimulating academic environment and for her support in facilitating this internship opportunity.

I extend my heartfelt thanks to Dr. N. M. Kanhe, Principal, Bajaj Institute of Technology, Wardha, for his visionary leadership and for maintaining the high academic standards of the institution that inspire students to pursue excellence in technical endeavors.

I also wish to thank Kiran Academy, Pune, for providing me with the platform, resources, and real-world problem statement that served as the foundation of this internship project.

The exposure to professional software development practices during this internship has been a transformative learning experience.

Date:

Place: Wardha

Sr. No.	Name of the Student	PRN Number	Signature
1	Sanket Prajapati	23046491245506	

ABSTRACT

FoodBridge is a full-stack food rescue web application developed during a six-month internship at Kiran Academy, Pune, from December 2025 to May 2026. The platform addresses the pervasive problem of food waste by creating a digital bridge between food donors — including restaurants, event organizers, and households — and verified Non-Governmental Organizations (NGOs) engaged in feeding underprivileged communities. The system is built on a decoupled architecture comprising a Django 5.0 REST API backend and a React 19 with Vite 8 single-page application frontend. The backend manages user authentication via JSON Web Tokens (JWT), role-based access control for three distinct user types (Donor, NGO, and Administrator), geospatial distance computation using the Haversine formula integrated with the Geopy Nominatone geocoding service, and an automated email notification system powered by Gmail SMTP. Food images are stored on Cloudinary, and the production database is hosted on Railway PostgreSQL. The core business workflow of FoodBridge operates in three stages: a donor posts surplus food with photographs, location, and a pickup window; a verified NGO within the configured service radius claims the donation; and at the time of physical pickup, the NGO presents a system-generated six-digit One-Time Password (OTP) to the donor, who verifies it to confirm the handover. This OTP-based pickup verification mechanism eliminates fraudulent claims and ensures accountability. Additional features include in-application notifications, an administrator dashboard for NGO verification, automated expiry of stale donations, and a Leaflet-powered interactive route map. The completed platform was successfully deployed with the frontend on Vercel and the backend on Render, demonstrating a resource-efficient, production-ready food rescue solution.

Keywords: Food Rescue, Django REST Framework, React, JWT Authentication, OTP Verification, Haversine Formula, NGO, Food Waste Reduction, Cloudinary, PostgreSQL.

TABLE OF CONTENTS

Caption	Page No.
Certificates	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Figures	viii
List of Tables	ix
Abbreviations	x
Chapter 1: Plan of Internship Program	11
1.1 Introduction and Area of Internship	11
1.2 Duration of Internship	12
Chapter 2: Overview of the Organization	14
2.1 Brief History of Kiran Academy	14
2.2 Introduction and Working Domains	14
2.3 Company Clients	15
2.4 Main Offices	16
2.5 Departments of the Organization	16
Chapter 3: Training Program	18
3.1 Training Modules and Duration	18
3.2 Tasks Assigned During Internship	20
Chapter 4: Weekly Overview of Internship Activities	23
Chapter 5: Projects Completed	30

5.1 Introduction to FoodBridge	30
5.2 Problem Statement	31
5.3 Literature Survey and Technologies Learnt	32
5.4 Proposed Methodology — System Architecture and Process Flows	35
5.5 System Requirement Specifications	42
5.6 Implementation — Modules, Screenshots, and Features	44
Chapter 6: Conclusions and Summary	58
6.1 Conclusions and Inferences	58
6.2 Lessons Learnt	59
References	61
Annexure A — Deployment Configuration	63
Annexure B – Industry Mentor Document Verification	64

LIST OF FIGURES

Figure No.	Caption	Page No.
Fig. 5.1	FoodBridge Landing Page — Hero Section	31
Fig. 5.2	System Architecture Diagram — FoodBridge Platform	36
Fig. 5.3	Entity Relationship Diagram – FoodBridge Application	40
Fig. 5.4	Donor User Profile Page	45
Fig. 5.5	Donation Detail Page with CLAIMED Status and Countdown Timer	45
Fig. 5.6	My Donations Page — OTP Input Field (Empty)	46
Fig. 5.7	My Donations Page — OTP Entered for Verification	46
Fig. 5.8	My Donations Page — Donation Marked as PICKED UP	47
Fig. 5.9	NGO Organisation Profile — Verified Status and Location Details	48
Fig. 5.10	NGO User Profile — Account Details and Email Verification	48
Fig. 5.11	My Food Claims — CLAIMED Status with OTP Display	49
Fig. 5.12	My Food Claims — Donor Contact Information Revealed	50
Fig. 5.13	Pickup Route Map — Leaflet Interactive Map with OSM Tiles	50
Fig. 5.14	My Food Claims — PICKED UP Confirmation Status	51
Fig. 5.15	NGO Notifications — Organization Verification Alert	51
Fig. 5.16	Donor Notifications — Donation Claimed by NGO	52
Fig. 5.17	Donor Notifications — Pickup Completion Confirmation	52
Fig. 5.18	Email — Verify Email Address (Registration Flow)	54
Fig. 5.19	Email — NGO Receives Pickup OTP Code	55
Fig. 5.20	Email — Donor Notified of Food Claim with NGO Contact	56
Fig. 5.21	Email — Donor Contact Details Sent to NGO with Pickup Address	57

LIST OF TABLES

Table No.	Caption	Page No.
Table 1.1	Internship Duration Summary	12
Table 3.1	Training Modules and Technologies Covered	18
Table 4.1	Weekly Overview of Internship Activities	23
Table 5.1	Comparison of Existing Food Rescue Platforms	34
Table 5.2	Technology Stack Summary	37
Table 5.3	Database Models and Key Fields	41
Table 5.4	Hardware and Software Requirements	42
Table 5.5	Backend API Endpoints — Public	42
Table 5.6	Backend API Endpoints — Authenticated	43
Table 5.7	Backend API Endpoints — Admin Only	43
Table 5.8	Frontend Application Routes	44

ABBREVIATIONS

<i>Abbreviation</i>	Full Form
<i>API</i>	Application Programming Interface
<i>ASGI</i>	Asynchronous Server Gateway Interface
<i>CORS</i>	Cross-Origin Resource Sharing
<i>CRUD</i>	Create, Read, Update, Delete
<i>DRF</i>	Django REST Framework
<i>ER</i>	Entity-Relationship
<i>HTML</i>	Hypertext Markup Language
<i>HTTP</i>	Hypertext Transfer Protocol
<i>HTTPS</i>	Hypertext Transfer Protocol Secure
<i>JWT</i>	JSON Web Token
<i>NGO</i>	Non-Governmental Organization
<i>OSM</i>	OpenStreetMap
<i>OTP</i>	One-Time Password
<i>PRN</i>	Permanent Registration Number
<i>REST</i>	Representational State Transfer
<i>SPA</i>	Single-Page Application
<i>SMTP</i>	Simple Mail Transfer Protocol
<i>SQL</i>	Structured Query Language
<i>SQLite</i>	Self-contained SQL database engine
<i>UI</i>	User Interface
<i>URL</i>	Uniform Resource Locator
<i>VITE</i>	Next-Generation Frontend Build Tool

CHAPTER 1

PLAN OF INTERNSHIP PROGRAM

1.1 Introduction and Area of the Organization

The internship for the final year of the Bachelor of Technology programmed in Computer Engineering was undertaken at Kiran Academy, Pune, Maharashtra, from 01 December 2025 to 18 May 2026, spanning a duration of approximately five and a half months. Kiran Academy is a professional software training and development organization located in Pune, Maharashtra, India. The organization specializes in providing end-to-end software development training, full-stack web application development services, and mentoring aspiring software engineers across a broad spectrum of modern technologies. The primary domain of the organization encompasses web-based application development, cloud computing, database management, and software engineering best practices, with a strong focus on equipping trainees with widely adopted skills that are directly applicable in professional environments.

The internship was structured to provide hands-on, project-based learning in modern full-stack web development technologies, specifically the Django web framework for backend development, the React JavaScript library for frontend development, and a suite of associated cloud and DevOps tools including Cloudinary for media storage, Railway for database and backend hosting, Vercel for frontend deployment, and SendGrid for transactional email services. The internship culminated in the independent design, development, and deployment of a complete, fully functional and deployable web application titled FoodBridge — a food rescue platform that connects food donors with verified Non-Governmental Organizations (NGOs) to reduce food waste in urban areas.

The area of internship was located within the software development and training division of Kiran Academy. During the internship, the student was assigned to the web development training module, where structured sessions on Python, Django, JavaScript, React, REST API design, database modelling, authentication systems, and deployment pipelines were conducted. The mentorship and technical guidance were provided by Ms. Prajakta V

Murmure, the designated Industry Mentor, who supervised the progress of the FoodBridge project from its inception through to its final production deployment. The faculty guide for this internship at the institutional level was Dr. V. R. Palekar, Department of Computer Engineering, Bajaj Institute of Technology, Wardha.

The internship was aligned with the technical and professional objectives of the final year B.Tech. curriculum, offering the student an opportunity to apply theoretical knowledge acquired during the course of study to the solution of a real-world problem of societal significance. Food waste is a global challenge of considerable magnitude; according to the Food and Agriculture Organization of the United Nations (FAO), approximately one-third of all food produced globally for human consumption is lost or wasted every year. In the Indian context, this problem is particularly acute in urban areas where restaurants, weddings, and large community gatherings generate significant quantities of surplus food, while simultaneously, millions of people lack access to adequate nutrition. Food Bridge was conceived as a digital solution to bridge this gap by enabling rapid and verifiable transfer of surplus food from donors to NGOs engaged in community feeding programmes.

1.2 Duration of Internship

The internship was formally conducted from 01 December 2025 to 18 May 2026, as certified by Kiran Academy, Pune. The total duration of the internship was approximately twenty-four weeks, encompassing all phases of the software development lifecycle, including requirements analysis, system design, iterative implementation, testing, deployment, and documentation. Table 1.1 provides a summary of the internship duration.

Table 1.1: Internship Duration Summary

Detail	Information
Internship Start Date	01 December 2025
Internship End Date	18 May 2026
Total Duration	Approximately 24 Weeks (~170 Working Days)
Organization	Kiran Academy, Pune, Maharashtra
Industry Mentor	Ms. Prajakta V Murmure
Faculty Guide	Dr. V. R. Palekar
Mode	Hybrid (Remote)

The internship was conducted in a progressive manner, moving from foundational training in the first six weeks through increasingly complex implementation tasks in the subsequent weeks. The final weeks of the internship were dedicated to production deployment, performance optimization, and the preparation of this thorough internship report. The internship certificate, as issued by Kiran Academy upon successful completion, is appended to this report as per the prescribed format.

CHAPTER 2

OVERVIEW OF THE ORGANISATION

2.1 Brief History of Kiran Academy

Kiran Academy is a professional technology training and software development organization headquartered in Pune, Maharashtra, India. The academy was established with the mission of bridging the gap between academic knowledge and industry expectations in the field of information technology and software engineering. Over the years, Kiran Academy has developed a reputation for providing rigorous, project-based training programmed that equip students and early-career professionals with the skills required to succeed in the competitive technology industry. The organization has mentored a significant number of engineering graduates from universities across Maharashtra, helping them transition into professional software development roles at organizations ranging from startups to established enterprises.

The academy has grown steadily since its founding, expanding its curriculum to encompass contemporary technologies such as Python-based web development, React.js, Node.js, cloud computing platforms, database management systems, and DevOps practices. Kiran Academy places a strong emphasis on practical learning, requiring all trainees to develop and deploy real-world projects as a core component of their training programmed. This approach ensures that graduates of the academy's programmed are job-ready from the first day of their professional careers. The organization is led by experienced software industry professionals and educators who bring a wealth of practical knowledge to the training environment.

2.2 Introduction of the Organization — Working Domains, Products, and Services Offered

Kiran Academy operates across several interconnected working domains within the information technology sector. The primary domain is software development training, wherein the organization provides structured, curriculum-driven programmed covering the complete software development lifecycle. These programmed span frontend development technologies including HTML5, CSS3, JavaScript, React.js, and Vue.js; backend development technologies including Python, Django, Django REST Framework, Node.js,

and Express.js; database management systems including MySQL, PostgreSQL, and MongoDB; and cloud and deployment platforms including AWS, Railway, Render, Vercel, and Heroku.

A secondary domain of the organization is software development consulting and project delivery, wherein Kiran Academy's technical team undertakes the development of web applications for client organizations. This includes the design of system architectures, implementation of RESTful APIs, development of responsive user interfaces, and deployment of fully operational applications on cloud infrastructure. The consulting arm of the organization provides trainees with exposure to real client engagements, enhancing their understanding of professional software development workflows.

The services offered by Kiran Academy include full-stack web development bootcamps, one-on-one mentoring sessions, project portfolio development for placement candidates, and corporate training programmed for organizations seeking to upskill their technical workforce. The organization also provides placement assistance to successful graduates of its training programmed, leveraging its network of industry connections in the Maharashtra technology ecosystem.

2.3 Company Clients

As a training and consulting organization, Kiran Academy serves a diverse client base. The primary clients of the academy's training services are engineering students from various universities in Maharashtra, particularly those in the final years of their B.Tech. and B.E. programmes in Computer Engineering, Information Technology, and Electronics and Computer Science. These students enroll in the academy's programmes to supplement their academic education with practical, industry-oriented skills that enhance their employability. Additionally, the academy serves working professionals seeking to upskill or transition into software development roles.

On the consulting side, the organization has worked with small and medium-sized enterprises in the Pune region requiring web application development services. These clients' span sectors including education, retail, and social services. The organization's focus on open-source technologies and multi-service cloud-deployed deployment strategies makes its services accessible to

organizations operating with limited infrastructure budgets. Specific client engagement details remain confidential in accordance with the organization's data privacy policies.

2.4 Main Offices

Kiran Academy's primary office and training center is located in Pune, Maharashtra, India. Pune is a major hub for the Indian technology and education industry, home to numerous software companies, engineering colleges, and training institutes. The city's thriving technology ecosystem provides an ideal environment for Kiran Academy's operations, offering access to a large pool of engineering graduates, industry professionals, and technology events. The organisation's training sessions and mentoring activities are conducted primarily from this Pune location, with remote and hybrid engagement options available to trainees who are unable to attend on-site sessions.

2.5 Departments of the Organization

Kiran Academy is organized into several functional departments that collectively support its training and development operations. The Training and Curriculum Department is responsible for the design, development, and continuous updating of the training programmes offered by the academy. This department ensures that the curriculum remains aligned with current industry demands and incorporates emerging technologies as they gain adoption in the professional market.

The Mentoring and Project Guidance Department provides one-on-one and group mentoring sessions to trainees. Industry mentors assigned to this department, including Ms. Prajakta V Murmure who served as the industry mentor for this internship, supervise the progress of trainee projects, provide technical feedback, conduct code reviews, and guide trainees through the challenges encountered during development. This department is central to the academy's project-based learning model.

The Technology and Development Department manages the organization's internal software development projects and consulting engagements. This department employs senior software developers who work on client projects and also serve as technical resources for training

sessions. The Placement and Career Services Department manages the academy's relationships with hiring companies and assists graduating trainees in securing employment through resume preparation, mock interviews, and referrals. Finally, the Administrative Department handles organisational operations including enrolment, scheduling, communications, and financial management.

CHAPTER 3

TRAINING PROGRAM

3.1 Training Modules, Duration, and Technologies Learnt

The training programme at Kiran Academy was structured as a series of progressive modules, each designed to build upon the knowledge and skills acquired in the preceding module. The training commenced with foundational sessions in Python programming and progressively advanced through backend web development, frontend development, database management, API design, cloud deployment, and finally project integration and delivery. Table 3.1 provides a systematic overview of the training modules, their durations, and the specific technologies covered within each module.

Table 3.1: Training Modules and Technologies Covered

Module No.	Module Name	Duration (Weeks)	Technologies / Tools Covered
M1	Python & OOP Fundamentals	2	Python 3.11, Classes, Decorators, File I/O, Exception Handling
M2	Django Backend Development	4	Django 5.0, Models, Views, URLs, Templates, Admin Panel, ORM
M3	Django REST Framework & APIs	3	DRF, Serializers, ViewSets, Token Auth, Pagination, Filtering
M4	JWT Authentication & Security	2	djangorestframework-simplejwt, Token Blacklist, CORS, Middleware
M5	Database Design & PostgreSQL	2	ER Diagrams, Migrations, SQLite (dev), PostgreSQL (prod), dj-database-url
M6	JavaScript & React Fundamentals	3	ES6+, React 19, Hooks, State, Props, React Router v7, Vite 8
M7	React Advanced Patterns	2	Context API, Custom Hooks, Protected Routes, Axios/Fetch wrappers
M8	Tailwind CSS & Responsive UI	1	Tailwind CSS 3.4, Responsive Design, Lucide React Icons
M9	Cloud Services & Email	1	Cloudinary, Gmail SMTP, SendGrid, python-dotenv, Environment Variables
M10	Production Deployment & DevOps	2	Railway, Render, Vercel, Gunicorn, WhiteNoise, Procfile, Health Checks
M11	Geospatial Features & Maps	1	Haversine Formula, Geopy Nominatim, Leaflet.js, OpenStreetMap
M12	Project Integration & Delivery	1	End-to-end integration, OTP system, Testing, Documentation

Each module of the training program was conducted through a combination of didactic instruction, live coding demonstrations by the industry mentor, and hands-on practical exercises. The first module — Python and Object-Oriented Programming Fundamentals — provided the essential foundation upon which all subsequent backend development work was built. Students were introduced to Python 3.11's core syntax, data structures, object-oriented principles including classes, inheritance, and polymorphism, as well as Python-specific features such as decorators, generators, and context managers.

The second and third modules focused on Django and the Django REST Framework, which constitute the primary backend technology stack of the FoodBridge application. Through these modules, the understanding of Django's Model-View-Template (MVT) architecture, the ORM-based approach to database management, URL routing, middleware, and the Django admin interface was thoroughly developed. The REST Framework module extended this knowledge to the design and implementation of RESTful APIs, covering serializers, class-based views, authentication mechanisms, and API testing methodologies.

The frontend development modules — Module 6 through Module 8 — introduced React 19 and the Vite build tooling ecosystem. These modules covered the fundamental building blocks of modern React development including functional components, the `useState` and `useEffect` hooks, React Router v7 for client-side navigation, and Tailwind CSS for utility-first responsive styling. Advanced React patterns covered in Module 7 included custom hooks, protected routing strategies, and the implementation of a reusable API fetch wrapper with automatic token attachment and error handling.

The final three modules of the training programme addressed the operational and deployment aspects of modern web application development. Module 9 covered cloud-based media storage using Cloudinary, transactional email delivery through Gmail SMTP and SendGrid, and environment variable management using `python-dotenv`. Module 10 provided detailed instruction in production deployment using Railway for the backend database and application hosting, Render as an alternative backend hosting platform, and Vercel for frontend deployment. Module 11 introduced geospatial computing concepts, specifically the Haversine formula for great-circle distance calculation between geographic coordinates,

address geocoding using the Geopy library with the Nominatim geocoder, and interactive map rendering using Leaflet.js with OpenStreetMap tiles.

3.2 Tasks Assigned During the Internship

The tasks assigned during the internship were directly aligned with the development of the FoodBridge web application. Each task represented a concrete deliverable within the overall project development roadmap established by the industry mentor at the commencement of the internship. The following paragraphs describe the principal tasks that were assigned and completed over the course of the internship.

Task 1 — Requirements Analysis and Project Planning: The internship began with a extensive requirements analysis phase. The problem statement of food waste and the need for a digital intermediary between donors and NGOs was studied in detail. User stories were developed for three categories of platform users: food donors, NGOs, and administrators. The key functional requirements identified included user registration and email verification, role-based access control, donation creation with photo upload, geospatial filtering of donations by distance, NGO verification workflow, OTP-based pickup confirmation, automated email notifications, and administrative oversight capabilities. Non-functional requirements such as scalability, security, and deployment cost optimization were also documented.

Task 2 — Database Schema Design: Following requirements analysis, the database schema was designed to capture all entities and relationships within the system. The primary entities identified were User, Organization (representing NGO profiles), Donation, DonationPhoto, Claim, Notification, and EmailLog. The relationships between these entities, including the one-to-one relationship between User and Organization, the one-to-one relationship between Donation and Claim, and the one-to-many relationship between Donation and DonationPhoto, were formalised in an entity-relationship diagram. The database schema was subsequently implemented as Django model classes.

Task 3 — Backend API Development: The bulk of the backend development work involved implementing the RESTful API endpoints that drive all platform functionality. This included the authentication endpoints for registration, login, logout, and email verification; the profile management endpoints for user and organization profiles; the donation management endpoints for creating, listing, viewing, updating, and cancelling donations; the claims management endpoints for claiming donations and verifying OTP codes; the notification management endpoints; and the administrative endpoints for NGO verification and email log review.

Task 4 — Frontend Application Development: The frontend application was developed as a React 19 single-page application using Vite 8. Eighteen distinct page components were implemented, covering all user-facing screens of the application. Reusable UI components including a multi-step form with step progress indicator, OTP input component, status badge, food type badge, skeleton loading cards, toast notification component, and address autocomplete were developed and integrated across multiple pages. The Leaflet-based interactive route map component for NGO pickup navigation was also implemented during this phase.

Task 5 — Email Notification System: The email notification system was implemented using Django's email framework with Gmail SMTP as the mail transport agent. HTML email templates were developed for four distinct notification scenarios: email address verification at registration, NGO approval notification, NGO rejection notification with reason, and contact exchange email at the time of donation claiming. The contact exchange emails deliver the donor's contact details and pickup instructions to the NGO, while simultaneously delivering the NGO's contact details and the six-digit OTP code to the donor. To prevent SMTP connection timeouts from causing request failures, the email sending operation was implemented in a background thread using Python's threading. Thread class.

Task 6 — Production Deployment: The final task involved deploying the complete FoodBridge application to production cloud infrastructure. The Django backend was deployed on Render using Gunicorn as the WSGI server and WhiteNoise for static file serving. The React frontend was deployed on Vercel with a single-page application rewrite rule configured in `vercel.json`. The production database was provisioned as a PostgreSQL instance on Railway, with the `DATABASE_URL` environment variable used to configure the backend's database connection. Cloudinary was configured for production media storage.

CHAPTER 4

WEEKLY OVERVIEW OF INTERNSHIP ACTIVITIES

This chapter provides a thorough week-by-week account of all activities, tasks, and learning outcomes achieved during the internship at Kiran Academy, Pune, from 01 December 2025 to 18 May 2026. Each entry describes the specific technical work undertaken, the modules completed, and the progress made on the FoodBridge project during the respective week.

Table 4.1: Weekly Overview of Internship Activities

Week	From – To	Topic / Module	Activities & Tasks	Technology	Module Status
Week 1	01 Dec – 07 Dec 2025	Orientation & Python Fundamentals	Introductory sessions on internship objectives and project overview. Revision of Python 3.11 syntax, data types, functions, and control flow. Introduction to Object-Oriented Programming: classes, <code>__init__</code> , inheritance, and encapsulation.	Python 3.11	M1 commenced
Week 2	08 Dec – 14 Dec 2025	Advanced Python & Django Introduction	Completed Python OOP with decorators, file I/O, exception handling, and virtual environments (venv). Introduction to Django framework: MVT architecture, project structure, <code>manage.py</code> , <code>settings.py</code> , and first "Hello World" Django view.	Python, Django 5.0	M1 completed; M2 commenced
Week 3	15 Dec – 21 Dec 2025	Django Models & ORM	Detailed study of Django ORM: model creation, migrations, field types, and relationships (ForeignKey, OneToOne, ManyToMany). Database queries using the ORM's filter, get, annotate, and aggregate methods. Django Admin panel customisation.	Django ORM, SQLite	M2 in progress
Week 4	22 Dec – 28 Dec	Django Views, URLs & Admin	Implementation of class-based and function-based views. URL routing	Django Views, Django Admin	M2 ~60% complete

	2025		configuration using path() and include(). Django Admin customisation with list_display, list_filter, and custom actions. Week concluded with a holiday on 25 Dec (Christmas).		
Week 5	29 Dec 2025 – 04 Jan 2026	Django REST Framework Introduction	Introduction to Django REST Framework (DRF). Implementation of ModelSerializer and HyperlinkedSerializer. APIView and generic views (ListAPIView, CreateAPIView). First REST endpoints for user listing tested with Postman. New Year holiday on 01 Jan 2026.	DRF, Serializers, Postman	M2 completed; M3 commenced
Week 6	05 Jan – 11 Jan 2026	DRF Authentication & JWT	Detailed study of DRF authentication backends. Implementation of JWT authentication using djangorestframework-simplejwt. Access token (24h) and refresh token (7d) configuration. Token blacklisting on logout. CORS configuration using django-cors-headers.	JWT, simplejwt, CORS	M3 & M4 in progress
Week 7	12 Jan – 18 Jan 2026	User Registration & Email Verification System	Designed and implemented the User model with role-based fields (DONOR/NGO/ADMIN). Implemented the /api/register/ endpoint with email verification token generation. Developed the HTML email template for verification emails. Tested end-to-end registration flow.	Django Email, HTML Templates	M4 completed; User model implemented
Week 8	19 Jan – 25 Jan 2026	Organisation Model & NGO Workflow	Designed the Organisation model (OneToOne with User). Implemented NGO profile creation, update, and retrieval endpoints. Implemented the admin NGO verification and rejection workflow with email notifications. Republic Day holiday on 26 Jan noted.	Django Models, Admin Actions	Organisation module complete

Week 9	26 Jan – 01 Feb 2026	Donation Model & CRUD APIs	Designed the Donation and DonationPhoto models. Implemented multipart form upload for food photos using Cloudinary storage. Developed all Donation CRUD endpoints: create, list, detail, update, and cancel. Implemented food type filtering.	Cloudinary, DRF, Multipart	Donation module backend ~75% complete
Week 10	02 Feb – 08 Feb 2026	Geospatial Features — Haversine & Geocoding	Implemented the haversine_distance() utility function in Python for geodesic distance computation. Integrated Geopy Nominatim for automatic address-to-coordinates geocoding with hierarchical fallback strategy. Distance-based filtering of donations by NGO service radius.	Geopy, Nominatim, Haversine Formula	M11 commenced & completed
Week 11	09 Feb – 15 Feb 2026	Claim System & OTP Implementation	Designed the Claim model with OneToOne relationship to Donation. Implemented auto-generation of 6-digit OTP on Claim.save(). Developed claim creation, my-claims list, and OTP verification endpoints. Implemented status transitions: AVAILABLE → CLAIMED → PICKED UP.	Django Models, OTP, Status FSM	Claims module backend complete
Week 12	16 Feb – 22 Feb 2026	Notification System & Email Logs	Implemented the Notification model and all notification CRUD endpoints (list, unread count, mark read, mark all read). Implemented the EmailLog model for tracking email delivery status (SENT/FAILED). Integrated notification creation into all key business flow events.	Notifications, EmailLog, DRF	Backend notification system complete
Week 13	23 Feb – 01 Mar 2026	Auto-Expiry & Background Cleanup Logic	Implemented stale claim detection logic: claims older than 45 minutes without OTP verification are automatically released, returning the donation to AVAILABLE status. Implemented donation	Python datetime, Django Q, Task Logic	M12 backend tasks complete

			auto-expiry when pickup window has elapsed. Integrated lazy cleanup execution on donation list load.		
Week 14	02 Mar – 08 Mar 2026	Admin Dashboard Backend & API Hardening	Implemented admin-specific API endpoints: platform statistics, pending NGO list, NGO verify/reject, and email logs. Added <code>is_admin()</code> permission class for admin-only routes. Extensive input validation using DRF serializer validators. Reviewed entire API for edge cases and error handling.	DRF Permissions, Validation	Backend API 100% complete
Week 15	09 Mar – 15 Mar 2026	React & Vite Setup — Frontend Foundation	Initialised the React 19 + Vite 8 frontend project. Configured React Router v7 with BrowserRouter and all application routes. Implemented the <code>api.js</code> utility (fetch wrapper with Bearer token attachment, 401 → /login redirect). Set up Tailwind CSS 3.4 and imported Lucide React icon library.	React 19, Vite 8, React Router v7, Tailwind CSS	M6 commenced; Frontend scaffold complete
Week 16	16 Mar – 22 Mar 2026	Core UI Components & Authentication Pages	Built the Navbar (role-aware), Footer, Layout wrapper, and PrivateRoute guard. Implemented the Login and Register pages with form validation and API integration. Implemented the VerifyEmail page for email token verification. Implemented the Home landing page with animated stats counters.	React Components, Tailwind, JWT	M7 in progress; Auth flow complete
Week 17	23 Mar – 29 Mar 2026	Donation Pages — 4-Step Form & Feed	Implemented the 4-step donation creation form (Details → Location → Photos → Review) with the StepProgress component. Implemented the DonationList page with food-type filtering and SkeletonCard loading states. Implemented the	React Multi-Step Form, Cloudinary Upload	M6 & M8 completed; Donation UI done

			DonationDetail page with countdown timer for pickup window.		
Week 18	30 Mar – 05 Apr 2026	My Donations, My Claims & OTP UI	Implemented the MyDonations page with OTP input and Verify & Complete button. Implemented the MyClaims page displaying the pickup OTP code, distance from NGO, donor contact information, and integrated Leaflet map. Implemented the custom OTPInput reusable component.	Leaflet.js, React State, Custom Hooks	M7 & M11 UI completed
Week 19	06 Apr – 12 Apr 2026	NGO Profile, Admin Dashboard & Notifications UI	Implemented OrgCreate and OrgProfile pages for NGO profile management. Implemented the AdminDashboard page with statistics cards and pending NGO verification controls. Implemented the Notifications and Activity pages. Implemented the AddressAutocomplete UI component.	React, Tailwind, Admin UI	M8 completed; All pages implemented
Week 20	13 Apr – 19 Apr 2026	Integration Testing & Bug Fixes	End-to-end integration testing of the complete Donor → NGO → Admin → OTP workflow. Identified and resolved issues including CORS header mismatches, JWT token refresh edge cases, Cloudinary upload failures on large images, and geocoding fallback failures. Dr. B. R. Ambedkar Jayanti holiday on 14 Apr.	Postman, Browser Dev Tools, Django Logs	Integration testing complete
Week 21	20 Apr – 26 Apr 2026	Production Deployment — Backend	Configured Gunicorn WSGI server and WhiteNoise for static files. Created Procfile and railway.toml deployment configurations. Provisioned PostgreSQL database on Railway. Ran production migrations. Configured all environment variables (SECRET_KEY, DATABASE_URL,	Railway, PostgreSQL, Gunicorn, WhiteNoise	M10 backend deployment complete

			Cloudinary credentials, SMTP credentials).		
Week 22	27 Apr – 03 May 2026	Production Deployment — Frontend & Email	Deployed React frontend on Vercel with SPA rewrite rule (vercel.json). Configured VITE_API_URL environment variable pointing to production backend. Resolved SMTP email delivery issues on Render by implementing background thread email sending. Tested production email flow end-to-end.	Vercel, Render, SendGrid, SMTP Threading	M10 frontend deployment complete
Week 23	04 May – 10 May 2026	Optimisation, Documentation & Report Drafting	Implemented Cloudinary auto-purge strategy plan for terminal-status donations. Compiled thorough project documentation (README.md, API reference). Commenced drafting of the internship report. Conducted final round of production smoke testing.	Documentation, Cloudinary Signals	Optimisation plan documented
Week 24	11 May – 18 May 2026	Final Report Completion & Submission	Completed all chapters of the internship report including Literature Survey, System Design, Implementation, and Conclusion. Integrated all screenshots and the system architecture diagram. Finalised and proofread the complete report document. Prepared submission package.	Report Writing, LaTeX Notation, Documentation	Project and report 100% complete

The weekly activities documented in Table 4.1 demonstrate a systematic and progressive approach to the development of the FoodBridge platform. The first thirteen weeks of the internship were devoted to training and backend development, ensuring a solid, tested API foundation before frontend development commenced. Weeks fourteen through nineteen were dedicated to building the complete React frontend application, integrating all backend APIs, and implementing the full suite of user interface components. The remaining weeks addressed production deployment, testing, optimisation, and the preparation of this report.

This structured progression ensured that all deliverables were completed on schedule within the internship timeline.

CHAPTER 5

PROJECTS COMPLETED

5.1 Introduction to FoodBridge

FoodBridge is a systematic food rescue web application developed as the primary project deliverable of this internship. The platform serves as a digital intermediary that enables food donors — comprising individual households, restaurants, event organizers, and other entities with surplus food — to post their available food donations on the platform, and enables verified Non-Governmental Organisations (NGOs) engaged in community feeding programmes to discover, claim, and collect those donations in a safe, accountable, and time-sensitive manner.

The application was conceived in response to the dual challenge of food waste and food insecurity that characterises modern urban societies. FoodBridge addresses this challenge by providing a structured, technology-mediated workflow that brings transparency, accountability, and efficiency to the food rescue process. Unlike informal food sharing arrangements, FoodBridge incorporates a formal NGO verification process to ensure that all recipient organizations are legitimate, an OTP-based pickup verification mechanism to confirm the physical handover of food, and an automated notification system to keep all parties informed at every stage of the process.

The application is built on a decoupled, API-first architecture. The backend is implemented using the Django web framework (version 5.0.3) and the Django REST Framework (DRF) for RESTful API design and delivery. The frontend is implemented as a React 19 single-page application (SPA) built with the Vite 8 bundler and styled using Tailwind CSS 3.4. Communication between the frontend and backend is mediated exclusively through JWT-authenticated REST API calls over HTTPS. This decoupled design ensures that the frontend and backend can be developed, tested, deployed, and scaled independently.

In its production configuration, the FoodBridge backend is deployed on the Render cloud platform, the frontend on Vercel, and the database on Railway PostgreSQL. Media files, specifically food photographs uploaded by donors, are stored on Cloudinary, a cloud-based

media management service. Transactional emails are delivered via Gmail SMTP. This multi-service cloud-deployed architecture strategy ensures that the platform operates reliably at scale while remaining cost-effective within the constraints of free-tier cloud service offerings.

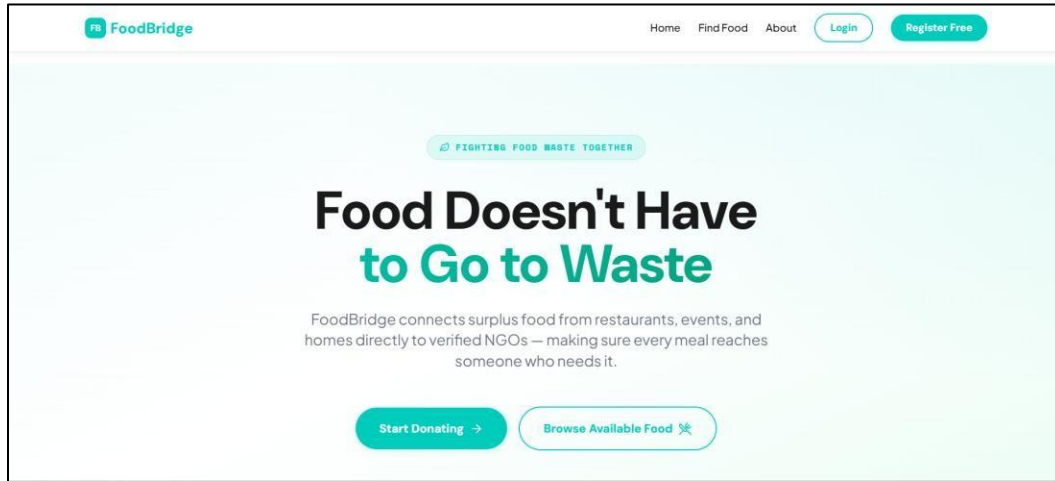


Fig. 5.1: FoodBridge Landing Page — Hero Section

Fig. 5.1 illustrates the landing page of the FoodBridge application as rendered in the production environment at `food-bridge-sandy.vercel.app`. The hero section prominently displays the platform's value proposition — "Food Doesn't Have to Go to Waste" — and provides two primary calls to action: "Start Donating" for food donors and "Browse Available Food" for NGOs. The navigation bar presents Login and Register Free options for unauthenticated visitors.

5.2 Problem Statement

Food waste is a global problem of staggering proportions. The United Nations Environment Programme (UNEP) and the FAO estimate that approximately 1.3 billion tonnes of food is wasted globally every year, representing one-third of all food produced for human consumption. In India, the National Sample Survey Organisation and other research bodies have documented significant post-consumer food waste at events, restaurants, and institutional canteens, while simultaneously, the National Family Health Survey (NFHS) has consistently revealed that a substantial proportion of the Indian population remains food insecure. This paradox — of concurrent food abundance and food insecurity — is a structural consequence of the absence of efficient mechanisms for redistributing surplus food.

In the context of urban Maharashtra, restaurants typically prepare food quantities in excess of demand to ensure service continuity, resulting in significant end-of-day surpluses. Wedding receptions, corporate events, and religious gatherings routinely produce food surpluses of considerable scale. Households frequently have perishable food items that cannot be consumed before their safe use-by date. In all these cases, the surplus food is often discarded, contributing to landfill waste and greenhouse gas emissions from decomposing organic matter, while organizations operating soup kitchens and community feeding programmes in the same geographic areas struggle with inconsistent food supplies.

The fundamental problem that FoodBridge was designed to solve can be stated as follows: there is no reliable, scalable, and accountable digital platform that enables food donors in Indian urban areas to connect with nearby verified NGOs and efficiently transfer surplus food to those organizations in a time-sensitive manner. Existing informal mechanisms — such as direct phone calls, social media posts, or WhatsApp groups — lack the structure, discoverability, and accountability required to operate at scale. They provide no mechanism for verifying the legitimacy of recipient organizations, no traceable record of food transfers, and no geographic filtering to ensure that food is claimed by organizations capable of collecting it promptly.

FoodBridge addresses this problem by providing a structured digital platform with the following key capabilities: a standardized donation listing workflow that captures all relevant information including food type, quantity, pickup location, and pickup time window; an administrative NGO verification process to ensure that only legitimate organizations can claim food donations; a geospatial distance-based filtering mechanism to ensure that donations are only visible to NGOs within a practical pickup radius; an OTP-based pickup confirmation system to ensure that every claim is physically verified; and a detailed email and in-application notification system to keep all parties informed throughout the process.

5.3 Literature Survey and Technologies Learnt

Prior to the design and development of FoodBridge, a systematic review of existing food rescue platforms and related academic literature was conducted to understand the state of the

art in digital food waste reduction systems, identify gaps in existing solutions, and inform the design decisions of the Food Bridge platform.

Feeding America's MealConnect platform is one of the most prominent food rescue technology solutions currently in operation in the United States. MealConnect enables food donors to post surplus food and connects them with local food banks and rescue organisations. The platform has facilitated the rescue of hundreds of millions of pounds of food since its inception. However, MealConnect is specifically designed for the North American food bank ecosystem and does not address the specific operational and regulatory context of Indian NGOs. Its architecture is not open or adaptable for localized deployment.

Food for Good, a social enterprise operating in India, operates a food rescue model in which volunteers collect surplus food from donors and deliver it to beneficiary organizations. While effective, this model is heavily dependent on volunteer availability and does not utilize digital technology to automate the matching and verification process. Similarly, Robin Hood Army and other volunteer-driven food rescue organizations in India rely primarily on WhatsApp coordination and do not have dedicated technology platforms with built-in donor-NGO matching, geographic filtering, or pickup verification capabilities.

Olio is a food sharing mobile application that enables individuals to share surplus food with neighbors. While successful in consumer contexts, Olio's model is designed for peer-to-peer sharing rather than the NGO-mediated bulk food rescue that FoodBridge targets. Too Good to Go is a commercial platform that enables consumers to purchase surplus food from restaurants at reduced prices, which represents a commercial rather than charitable redistribution model. Academic literature reviewed, including the work of Papargyropoulou et al. (2014) on the food waste hierarchy and Schanes et al. (2018) on food waste at the consumer level, provided theoretical grounding for the importance of redistribution as a food waste reduction strategy.

Table 5.1: Comparison of Existing Food Rescue Platforms

Platform	Model	Geographic Focus	NGO Verification	OTP / Pickup Confirm	open-source
MealConnect (Feeding America)	Donor → Food Bank	USA	Yes	No	No
Olio	Peer-to-Peer Sharing	UK/Global	No	No	No
Too Good To Go	Commercial Surplus	Europe/Global	N/A	QR Code	No
Robin Hood Army	Volunteer-Driven	India	Informal	No	No
Food for Good	Volunteer Pickup	India	Informal	No	No
FoodBridge (This Project)	Donor → NGO Digital	India (Urban)	Formal Admin-verified	6-digit OTP	Yes (Portfolio)

Table 5.1 clearly illustrates that FoodBridge occupies a distinctive position in the food rescue technology landscape. It is the only platform in this comparison that combines formal NGO verification, OTP-based pickup confirmation, geospatial distance filtering, and a locally deployable open-source architecture tailored for the Indian urban food rescue context.

The primary technologies learnt and applied during the development of FoodBridge are described below. Django 5.0 is a high-level Python web framework that follows the Model-View-Template (MVT) architectural pattern and adheres to the principles of convention over configuration and the Don't Repeat Yourself (DRY) paradigm. Django's built-in ORM provides database abstraction that enables developers to interact with relational databases using Python objects rather than raw SQL, while its migrations system manages schema changes in a version-controlled manner.

The Django REST Framework (DRF) is a powerful toolkit for building Web APIs on top of Django. DRF provides a rich set of abstractions including serializers for data validation and transformation, class-based views for HTTP method handling, authentication backends, permission classes, pagination, filtering, and throttling. The framework's browsable API interface was invaluable during the development and testing phase of the project.

JSON Web Tokens (JWT) constitute the authentication mechanism used in FoodBridge. A JWT is a compact, URL-safe means of representing claims between two parties as a JSON

object. In the context of Food Bridge, the `django-rest-framework-simplejwt` library was used to implement JWT-based authentication. Upon successful login, the server issues a signed access token (valid for 24 hours) and a refresh token (valid for 7 days). The mathematical basis of JWT security is the HMAC-SHA256 signing algorithm, which ensures that tokens cannot be tampered with without knowledge of the secret key, expressed as: `HMACSHA256(base64UrlEncode(header) + '.' + base64UrlEncode(payload), secret)`.

The Haversine formula is the mathematical algorithm used in FoodBridge to calculate the great-circle distance between two points on the surface of the Earth, given their geographic coordinates (latitude and longitude). The formula accounts for the spherical geometry of the Earth and is expressed as follows. Let ϕ_1, λ_1 and ϕ_2, λ_2 denote the latitudes and longitudes of the two points in radians respectively. The Haversine formula gives: $a = \sin^2((\phi_2 - \phi_1)/2) + \cos(\phi_1) * \cos(\phi_2) * \sin^2((\lambda_2 - \lambda_1)/2)$; $c = 2 * \text{atan2}(\sqrt{a}, \sqrt{1-a})$; $d = R * c$, where R is the Earth's mean radius of approximately 6,371 kilometres. In the FoodBridge implementation, this formula is computed in the `haversine_distance()` function within `backend/donations/utils.py` and is invoked during the claim creation process to verify that the claiming NGO's registered location falls within the configured service radius from the donation's pickup location.

React 19 is a JavaScript library for building user interfaces, developed and maintained by Meta. React follows a component-based architecture in which the UI is decomposed into reusable, stateful components. The unidirectional data flow model of React ensures predictable state management. Vite 8 is a next-generation frontend build tool that provides extremely fast development server startup times and optimized production builds through native ES module support and Rollup-based bundling. Tailwind CSS 3.4 is a utility-first CSS framework that enables rapid, consistent UI development by applying predefined utility classes directly in HTML/JSX markup.

5.4 Proposed Methodology — System Architecture, Modules, and Process Flows

The FoodBridge platform is built upon a three-tier, API-first architecture comprising a presentation tier (the React SPA frontend), a logic tier (the Django REST API backend), and a data tier (the PostgreSQL relational database). This decoupled architecture ensures clear

separation of concerns, independent scalability of each tier, and the ability to serve multiple client types — web browsers, mobile applications, or third-party integrations — from the same backend API without modification.

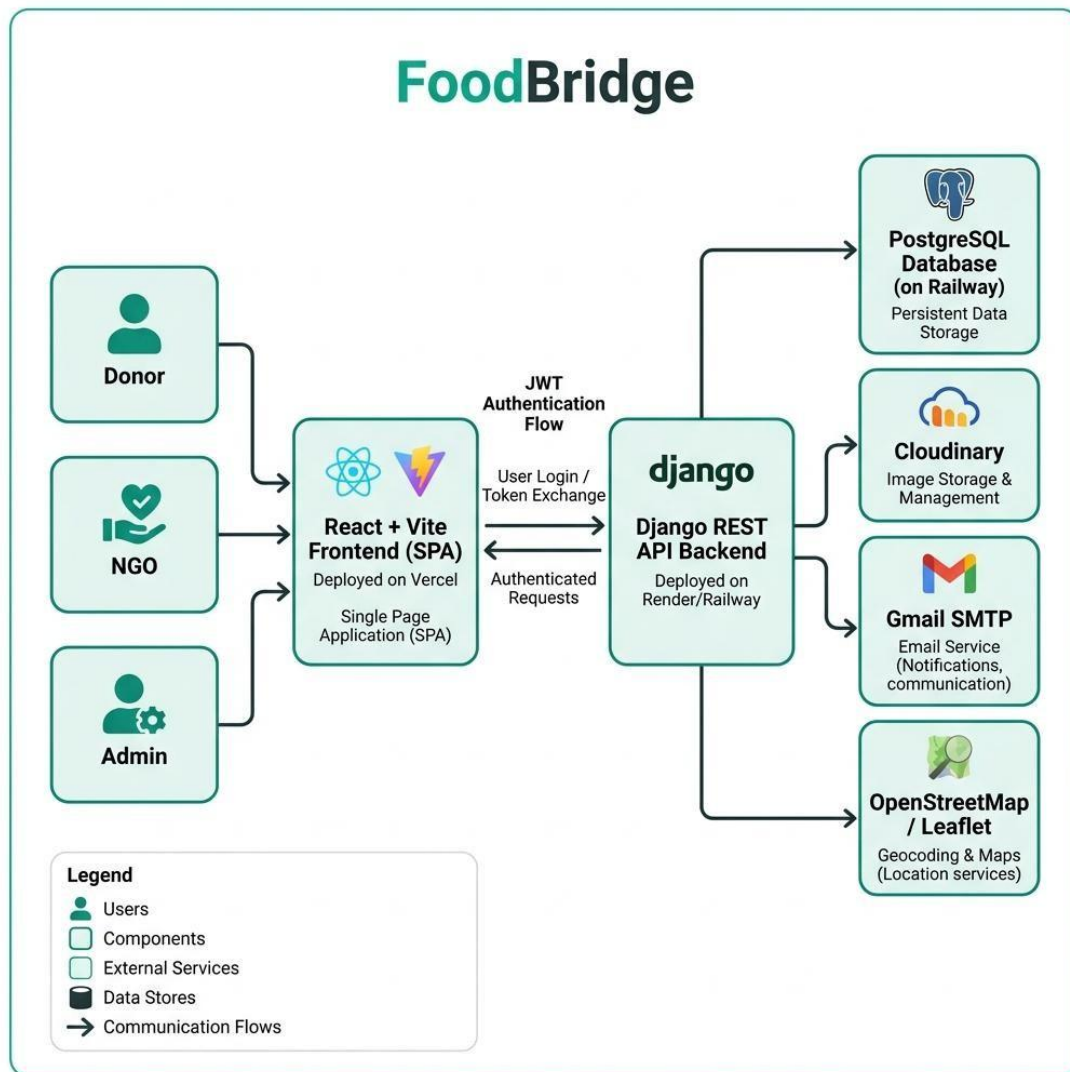


Fig. 5.2: System Architecture Diagram — FoodBridge Platform

Fig. 5.2 presents the extensive system architecture of the FoodBridge platform. Three categories of users — Donors, NGOs, and the Administrator — interact with the platform through the React frontend application deployed on Vercel. The frontend communicates with the Django REST API backend through HTTPS-secured RESTful API calls, with all authenticated requests carrying a JWT Bearer token in the Authorization header. The backend interfaces with the PostgreSQL database for persistent data storage,

Cloudinary for food photograph storage, Gmail SMTP for transactional email delivery, and Geopy Nominatim for address geocoding. The interactive Leaflet map on the NGO's My Claims page retrieves map tiles directly from OpenStreetMap servers.

Table 5.2: Technology Stack Summary

Layer	Technology	Version	Purpose
Backend Framework	Django	5.0.3	Core web framework, ORM, admin
REST API	Django REST Framework	3.17.1	API endpoints, serializers, auth
Authentication	django-rest-framework-simplejwt	5.5.1	JWT access/refresh token management
CORS	django-cors-headers	4.9.0	Cross-origin request handling
Database (Dev)	SQLite 3	3.x	Local development database
Database (Prod)	PostgreSQL (Railway)	15.x	Production relational database
Geocoding	Geopy (Nominatim)	2.4.1	Address to lat/lng conversion
Email	Gmail SMTP / SendGrid	—	Transactional email delivery
Media Storage	Cloudinary	1.42.2	Food photo cloud storage
Static Files	WhiteNoise	6.12.0	Efficient static file serving
Web Server	Gunicorn	25.3.0	Production WSGI server
Frontend Framework	React + Vite	19 + 8	SPA with fast HMR build tool
Styling	Tailwind CSS	3.4	Utility-first responsive UI
Routing (Frontend)	React Router	v7	Client-side SPA navigation
Maps	Leaflet + React-Leaflet	4.2.1	Interactive pickup route map
Icons	Lucide React	Latest	SVG icon components
Frontend Deployment	Vercel	—	Edge-deployed SPA hosting
Backend Deployment	Render	—	Backend + Gunicorn hosting

The FoodBridge platform is organised into two primary Django application modules: the core application and the donations application. The core application manages all user-related functionality including authentication, profile management, organisation (NGO) profiles, notifications, and email logging. The donations application manages the complete lifecycle of food donations, including donation creation, listing, claiming, OTP verification, and automated expiry.

5.4.1 Core Business Process Flows

FoodBridge implements five distinct business process flows that collectively cover the complete operational lifecycle of the platform.

Process Flow 1 — Donor Registration and Donation Posting: A user registers on the platform by submitting their email address, full name, phone number, password, and role selection (Donor). Upon registration, the system generates a cryptographically secure email verification token and dispatches a verification email containing a clickable link. The user activates their account by clicking the link, which calls the `/api/verify-email/` endpoint and marks the user's email as verified. The verified donor then logs in, receiving a JWT access token and refresh token pair. The donor navigates to the `/donate` route and completes the four-step donation form: Step 1 captures the food title, description, type (VEG/NON_VEG/VEGAN/MIXED), quantity in servings, and pickup window start and end times. Step 2 captures the pickup address, which is automatically geocoded by the backend to latitude and longitude coordinates using Geopy Nominatim. Step 3 allows the donor to upload one or more food photographs, which are stored on Cloudinary. Step 4 presents a review of all entered information before final submission. Upon submission, the donation is created with a status of AVAILABLE.

Process Flow 2 — NGO Registration and Verification: An NGO registers on the platform with the role selection of NGO. After email verification, the NGO user creates an organisation profile at `/org/create`, providing the organisation name, registration number, address, and service radius in kilometres. The address is geocoded to coordinates at the time of profile creation. Under the auto-verification policy implemented in FoodBridge, an NGO's organisation profile is automatically set to VERIFIED status upon email verification, enabling the NGO to immediately begin claiming food donations. An in-application notification and email notification are dispatched to confirm the verification. The administrator retains the ability to revoke verification or reject NGO applications through the administrative dashboard.

Process Flow 3 — Donation Claiming: A verified NGO browses available donations at `/donations`. The system's backend filters the donation list to exclude any donations whose pickup location falls outside the NGO's configured service radius, as determined by the Haversine distance calculation. When the NGO selects a donation and clicks the Claim for

My NGO button on the donation detail page, the backend creates a Claim record with a system-generated six-digit OTP code and transitions the donation's status to CLAIMED. Contact exchange emails are simultaneously dispatched: the NGO receives the donor's name, phone number, email address, pickup address, and the OTP code; the donor receives the NGO's name, phone number, and email address, along with instructions to await the OTP at pickup.

Process Flow 4 — OTP-Based Pickup Verification: At the time of physical food collection, the NGO representative presents the OTP code (visible on their My Claims page and in the email they received) to the donor. The donor navigates to their My Donations page, enters the six-digit code in the OTP input field, and clicks Verify and Complete. The backend validates the submitted OTP against the stored value for the corresponding Claim record. If the codes match, the Claim status is updated to COMPLETED and the Donation status is updated to PICKED_UP. A pickup completion notification is created for the donor, and a confirmation email is dispatched. If the codes do not match, an appropriate error response is returned and the claim remains in the CLAIMED state.

Process Flow 5 — Automated Expiry and Stale Claim Release: The FoodBridge system implements a lazy cleanup mechanism that is invoked each time the donation list endpoint is called. This mechanism performs two checks: first, it identifies any Claim records whose creation timestamp is more than 45 minutes in the past and whose associated donation status is still CLAIMED, indicating that no OTP verification has occurred within the expected pickup window. Such stale claims are released by deleting the Claim record and reverting the Donation status to AVAILABLE. Second, the mechanism identifies any donations whose configured pickup window end time has elapsed and whose status is still AVAILABLE or CLAIMED, and transitions those donations to EXPIRED status. Notifications are dispatched to the relevant donor and NGO in each case.

5.4.2 Database Schema

The FoodBridge database schema comprises six primary models, each implemented as a Django model class with a corresponding PostgreSQL table in the production environment. The User model extends Django's AbstractBaseUser and PermissionsMixin classes to provide a custom email-based authentication system in place of Django's default username-based authentication. The role field on the User model uses a ChoiceField with the values

DONOR, NGO, and ADMIN to implement role-based access control throughout the application.

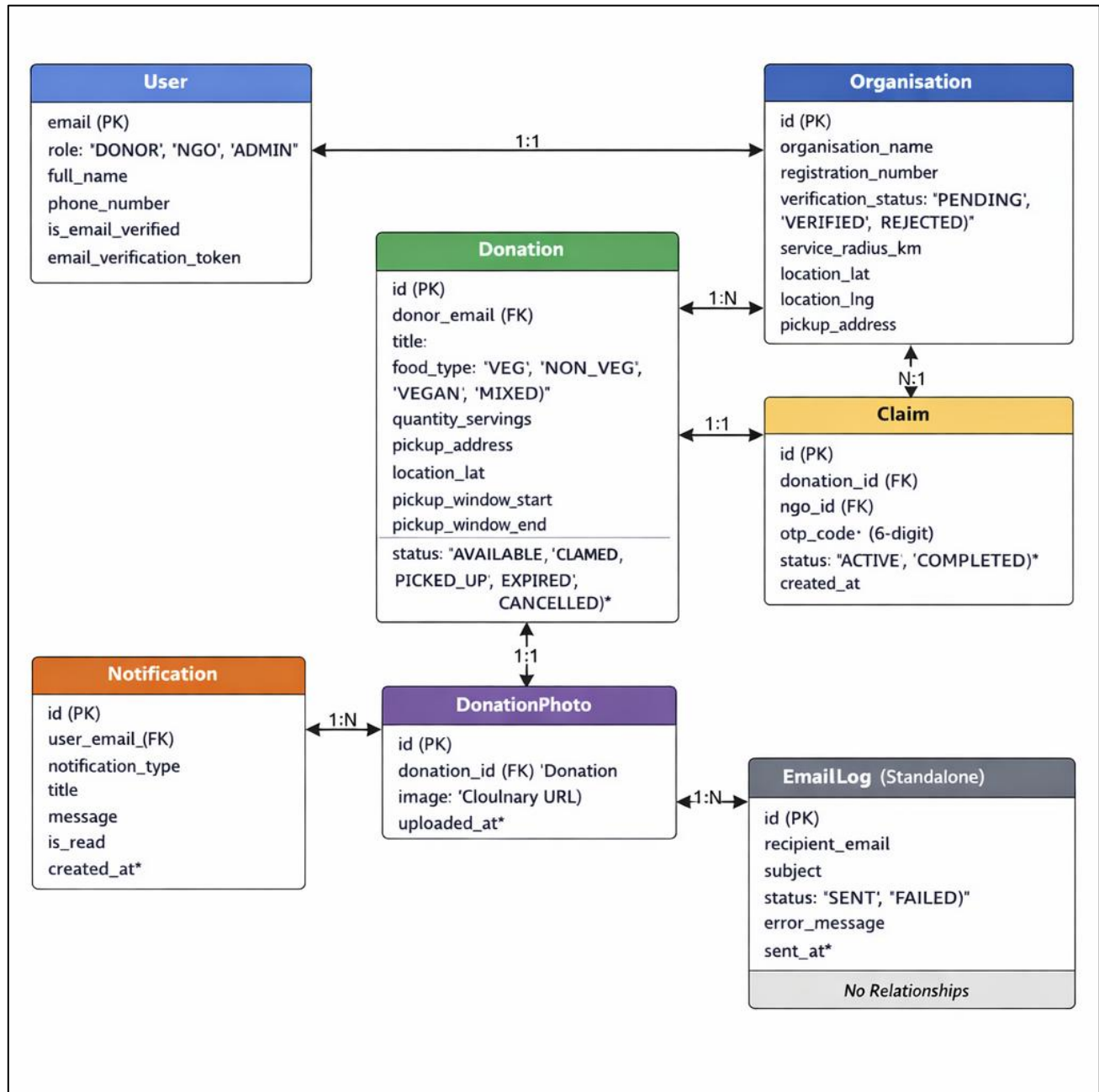


Fig. 5.3 ER-Diagram – FoodBridge Application

Table 5.3: Database Models and Key Fields

Model	Key Fields	Relationships
User	email (primary key), role (DONOR/NGO/ADMIN), full_name, phone_number, is_email_verified, email_verification_token	OneToOne → Organisation; ForeignKey ← Donation; ForeignKey ← Notification
Organisation	organisation_name, registration_number, verification_status (PENDING/VERIFIED/REJECTED), service_radius_km, location_lat, location_lng, pickup_address	OneToOne → User (ngo_user)
Donation	title, food_type (VEG/NON_VEG/VEGAN/MIXED), description, quantity_servings, pickup_address, location_lat, location_lng, pickup_window_start, pickup_window_end, status (AVAILABLE/CLAIMED/PICKED_UP/EXPIRED/CANCELLED)	ForeignKey → User (donor); OneToOne ← Claim; ForeignKey ← DonationPhoto
Claim	otp_code (auto-generated 6-digit), status (ACTIVE/COMPLETED), created_at	OneToOne → Donation; ForeignKey → Organisation (ngo)
DonationPhoto	image (Cloudinary URL), uploaded_at	ForeignKey → Donation
Notification	notification_type, title, message, is_read, created_at	ForeignKey → User (recipient)
EmailLog	recipient_email, subject, status (SENT/FAILED), error_message, sent_at	Standalone audit log

5.5 System Requirement Specifications

5.5.1 Hardware and Software Requirements

Table 5.4: Hardware and Software Requirements

Category	Requirement	Specification
Hardware	Development Machine	Intel Core i5 or equivalent, 8 GB RAM minimum, 256 GB SSD
Hardware	Processor Architecture	x86_64 (64-bit)
Hardware	Internet Connectivity	Broadband (for cloud API access during development)
Software	Operating System (Dev)	Windows 10/11, macOS 12+, or Ubuntu 20.04+
Software	Python Runtime	Python 3.11.x
Software	Node.js Runtime	Node.js 18+ (v20 or v24 recommended)
Software	Package Managers	pip (Python), npm 8+ (Node)
Software	Version Control	Git (any recent version)
Software	Code Editor	Visual Studio Code with Python and ESLint extensions
Software	API Testing	Postman or Thunder Client (VS Code extension)
Software	Browser	Google Chrome 120+ or Mozilla Firefox 120+
Cloud	Backend Hosting	Render (free tier: 512 MB RAM, shared CPU)
Cloud	Frontend Hosting	Vercel (free tier: unlimited static deployments)
Cloud	Database	Railway PostgreSQL (free tier: 500 MB storage)
Cloud	Media Storage	Cloudinary (free tier: 25 GB storage, 25 GB bandwidth/month)
Cloud	Email Service	Gmail SMTP (500 emails/day) / SendGrid (100 emails/day free)

5.5.2 API Endpoint Specifications

The FoodBridge backend exposes a thorough RESTful API comprising public endpoints accessible without authentication and protected endpoints requiring a valid JWT Bearer token. Tables 5.3, 5.4, and 5.5 enumerate all API endpoints implemented in the system.

Table 5.5: Backend API Endpoints — Public (No Authentication Required)

Method	Endpoint	Description
GET	/api/health/	Platform health check — returns {status: 'ok'}
GET	/api/stats/	Platform statistics: total donations, NGOs, pickups
POST	/api/register/	Register new user (Donor or NGO), returns JWT tokens
POST	/api/login/	User login, returns JWT access and refresh tokens
GET	/api/verify-email/?token=...	Verify email address via token from verification email
GET	/api/donations/	List all AVAILABLE donations (filterable by food_type, lat, lng)
GET	/api/donations/<id>/	Retrieve details of a single donation

Table 5.6: Backend API Endpoints — Authenticated (Bearer Token Required)

Method	Endpoint	Description
POST	/api/logout/	Logout — blacklists the JWT refresh token
GET	/api/profile/	Retrieve the authenticated user's profile
PUT	/api/profile/update/	Update authenticated user's profile data
POST	/api/orgs/create/	Create NGO organisation profile
GET	/api/orgs/my/	Retrieve the authenticated NGO user's organisation
PUT	/api/orgs/update/	Update the authenticated NGO user's organisation
POST	/api/donations/create/	Create a new donation (Donor only, multipart with photos)
GET	/api/donations/my/	List all donations posted by the authenticated donor
PUT	/api/donations/<id>/update/	Update a donation posted by the authenticated donor
POST	/api/donations/<id>/cancel/	Cancel a donation (Donor only)
POST	/api/claims/create/	Claim an available donation (NGO only)
GET	/api/claims/my/	List all claims of the authenticated NGO
POST	/api/claims/<id>/verify-otp/	Verify pickup OTP (Donor only)
GET	/api/notifications/	List all notifications for the authenticated user
GET	/api/notifications/unread-count/	Count of unread notifications
PUT	/api/notifications/<id>/read/	Mark a specific notification as read
PUT	/api/notifications/read-all/	Mark all notifications as read

Table 5.7: Backend API Endpoints — Admin Only

Method	Endpoint	Description
GET	/api/admin/stats/	Retrieve platform-wide statistics for admin dashboard
GET	/api/admin/ngos/pending/	List all NGO organisations with PENDING verification status
POST	/api/admin/ngos/<id>/verify/	Approve an NGO organisation (sets status to VERIFIED)
POST	/api/admin/ngos/<id>/reject/	Reject an NGO organisation with a reason message
GET	/api/admin/email-logs/	Retrieve email delivery audit log

Table 5.8: Frontend Application Routes and Access Control

Route	Page / Component	Access Control
/	Landing Page (stats, hero, how it works)	Public
/about	About FoodBridge	Public
/register	User Registration (Donor / NGO)	Public
/login	JWT Login Page	Public
/verify-email	Email Token Verification	Public
/donations	Browse Available Food (filterable list)	Public
/donations/:id	Donation Detail + Claim Button	Public
/donate	4-Step Donation Creation Form	Donor Only
/my-donations	Donor's Donations + OTP Verify	Donor Only
/my-claims	NGO's Claims + OTP + Leaflet Map	NGO Only
/org/create	Create NGO Organisation Profile	NGO Only
/org/profile	Edit NGO Organisation Profile	NGO Only
/profile	User Profile Page	Authenticated
/activity	Activity / History Page	Authenticated
/notifications	Notification Centre	Authenticated
/foodbridgeapplication/admin	Admin Login Page	Public
/foodbridgeapplication/admin/dashboard	Admin Dashboard	Admin Only

5.6 Implementation — Modules, Screenshots, and Features

This section presents the implemented FoodBridge web application through annotated screenshots of each major module, demonstrating the complete Donor and NGO workflows as executed in the production deployment at food-bridge-sandy.vercel.app.

5.6.1 Donor Module

The Donor module encompasses all functionality available to a registered and email-verified donor on the FoodBridge platform. Fig. 5.3 illustrates the Donor Profile page, which displays the authenticated donor's account details including email address, registered phone number, membership duration, and email verification status. The role badge clearly indicates the user's Donor role, and the interface provides logout functionality and profile editing capabilities.

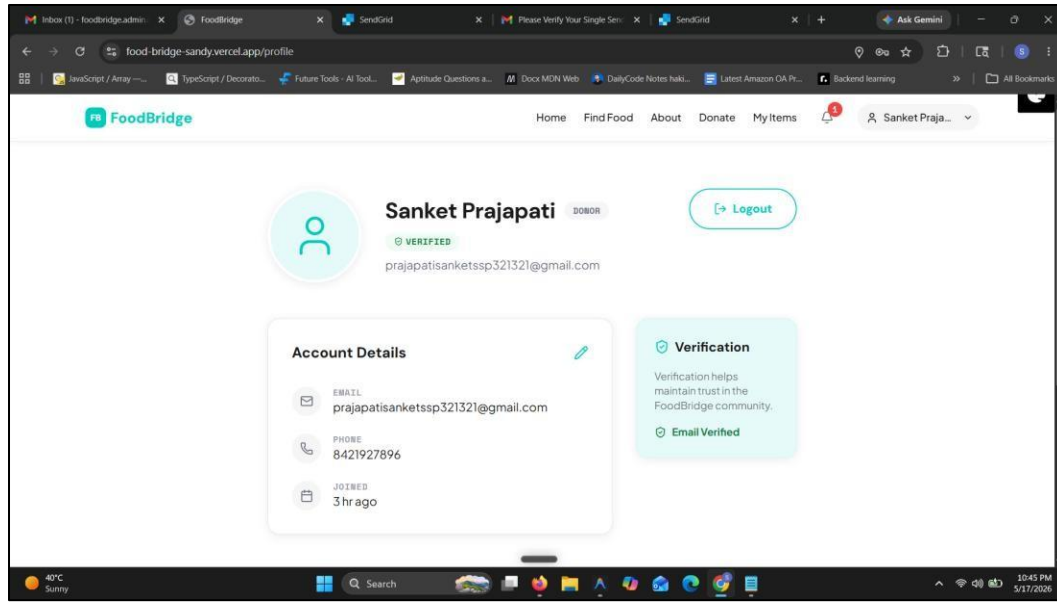


Fig. 5.4: Donor User Profile Page Showing Account Details and Email Verification Status

Fig. 5.4 presents the Donation Detail page for a food item titled 'Biryani Pulao', displaying the food photograph, food type badge (VEG), donation identifier (DON-00001), quantity (45 servings), donor name (Sanket Prajapati), pickup location (Ramdaspath, Nagpur), and a real-time countdown timer showing the remaining time within the pickup window. The CLAIMED status badge indicates that an NGO has already claimed this donation.

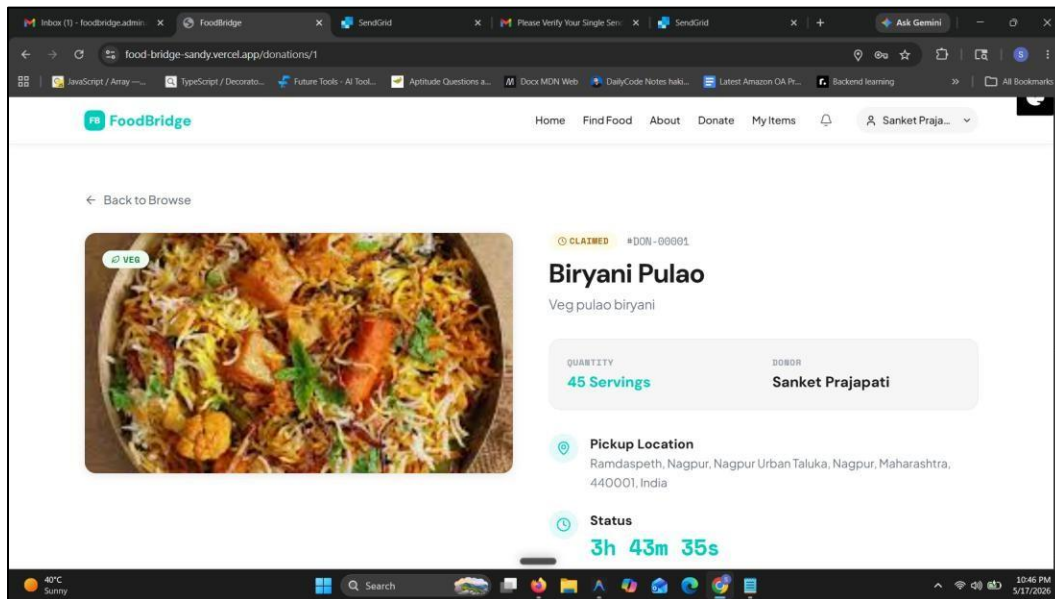


Fig. 5.5: Donation Detail Page — Biryani Pulao (VEG, 45 Servings) with CLAIMED Status and Pickup Countdown

The My Donations page, shown in Fig. 5.5 and Fig. 5.6, provides the donor with a dashboard to manage their posted donations and verify pickup OTPs. Fig. 5.5 shows the page with an empty OTP input field on a CLAIMED donation, while Fig. 5.6 shows the state after the donor has entered the six-digit OTP code (414921) received from the NGO. The 'Verify and Complete' button, when clicked, submits the OTP to the backend for validation.

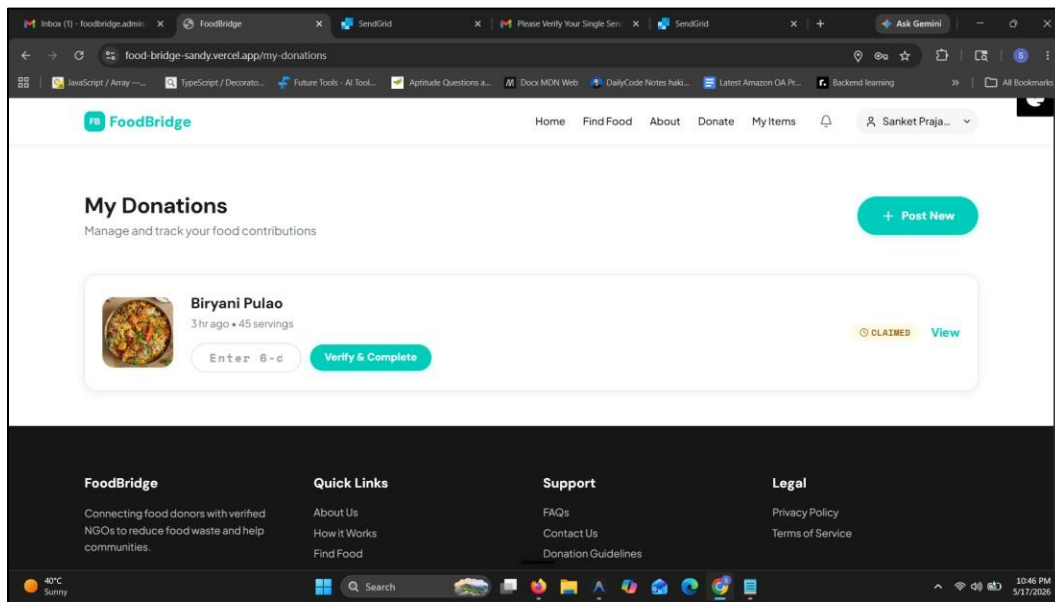


Fig. 5.6: My Donations Page — OTP Input Field Empty; CLAIMED Status Indicating Pending Verification

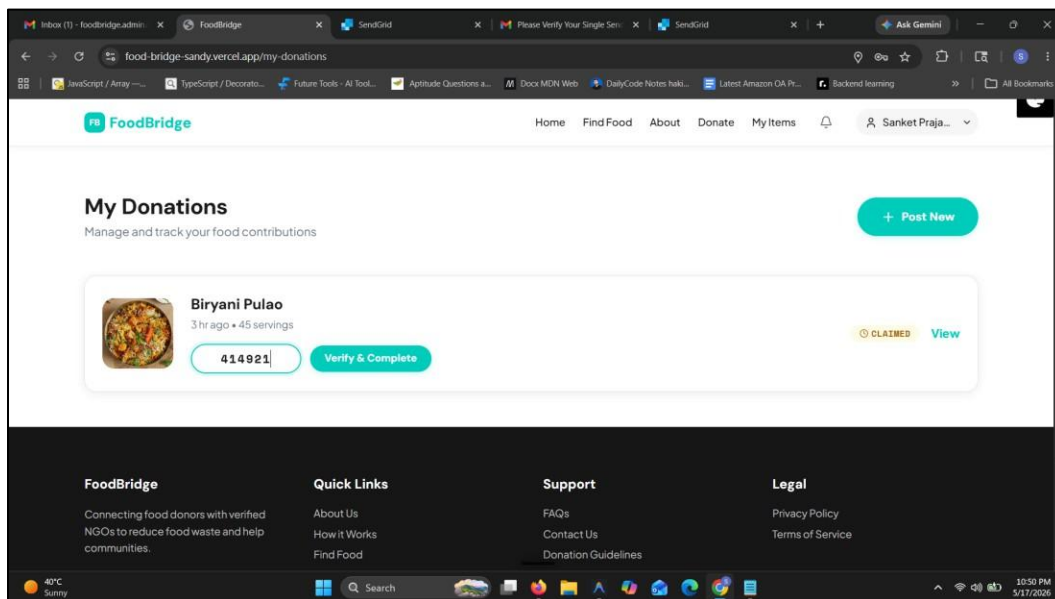


Fig. 5.7: My Donations Page — OTP Code 414921 Entered; Ready for Pickup Verification

Fig. 5.7 demonstrates the state of the My Donations page after successful OTP verification. The donation's status badge has been updated to PICKED UP, confirming that the physical food handover has been completed and recorded in the system. A 'Pickup verified!' toast notification is visible in the bottom-right corner of the interface, providing immediate user feedback upon successful OTP validation.

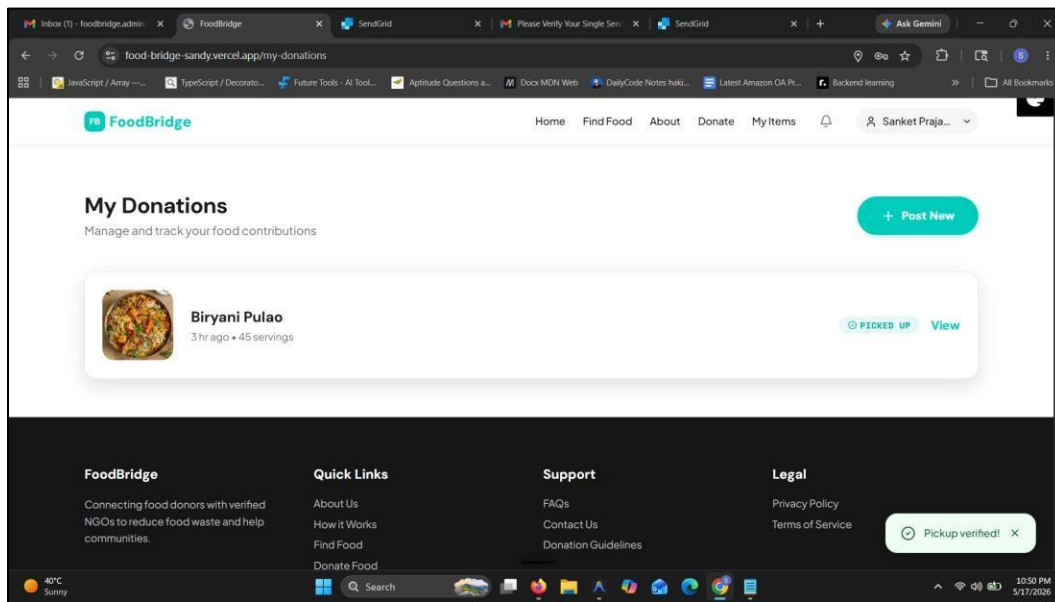


Fig. 5.8: My Donations Page — Donation Status Updated to PICKED UP with Confirmation Toast

5.6.2 NGO Module

The NGO module provides verified non-governmental organisations with the capability to browse and claim available food donations, manage their organisation profile, view claimed donation details and OTP codes, and access interactive pickup route maps. Fig. 5.8 illustrates the NGO Organisation Profile page for the test NGO 'Sadhguru Trust', showing the organisation's name, registration number (NGO-1234), VERIFIED status badge, pickup address in Ramdaspath, Nagpur, geographic coordinates (21.136585, 79.074987), and configured service radius of 29 kilometres.

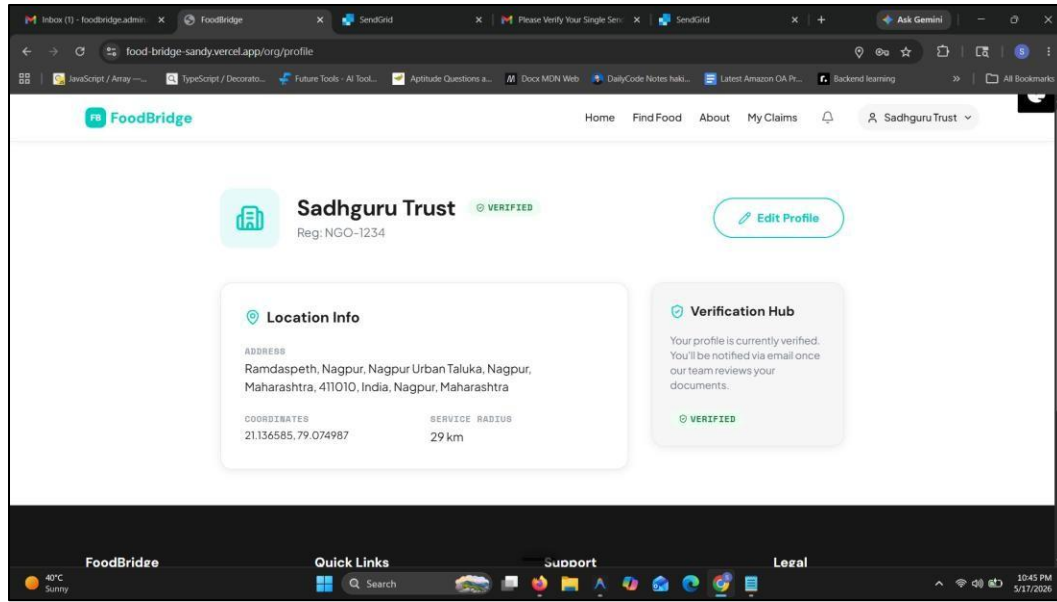


Fig. 5.9: NGO Organisation Profile Page — Sadhguru Trust (VERIFIED, 29 km Service Radius)

Fig. 5.9 shows the NGO user account profile page for the representative of Sadhguru Trust, displaying the account details including email address, registered phone number, and membership duration. The Verification panel confirms the user's email verified status, and a 'Manage NGO Profile' button provides quick navigation to the organisation profile management page.

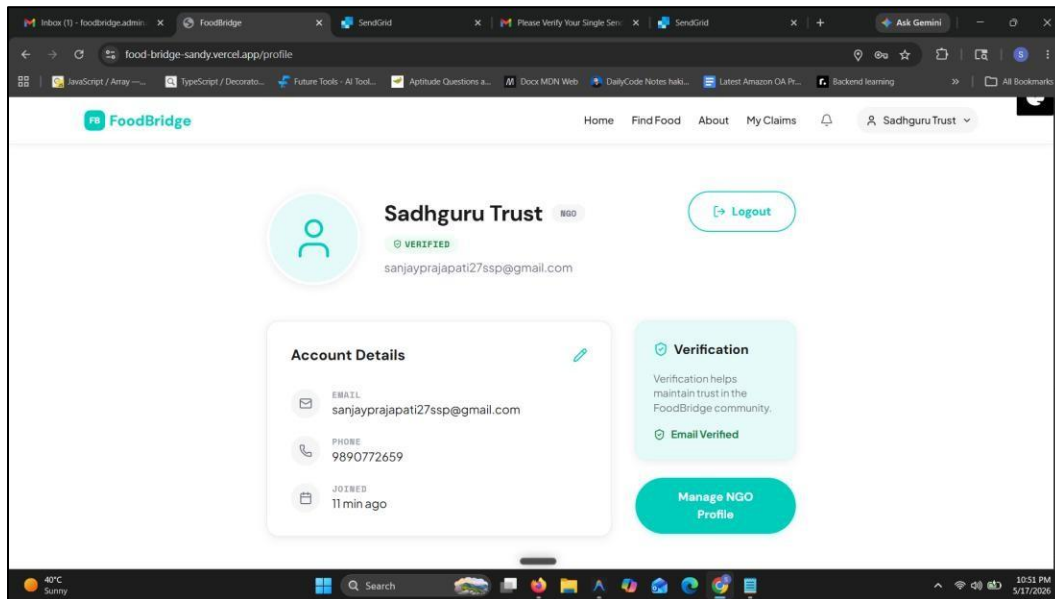


Fig. 5.10: NGO User Account Profile — Account Details and Email Verification Status

The My Food Claims page is the primary operational interface for NGO users. Fig. 5.10 shows the CLAIMED state of a food claim for 'Biryani Pulao' (Claim ID-0001). The left panel prominently displays the six-digit Pickup Code (414921), which the NGO representative must present to the donor at the time of collection. The claim card displays the distance from the NGO's registered location (1.85 km), the pickup address, and provides buttons to View Details, Show Contact, and Show Route.

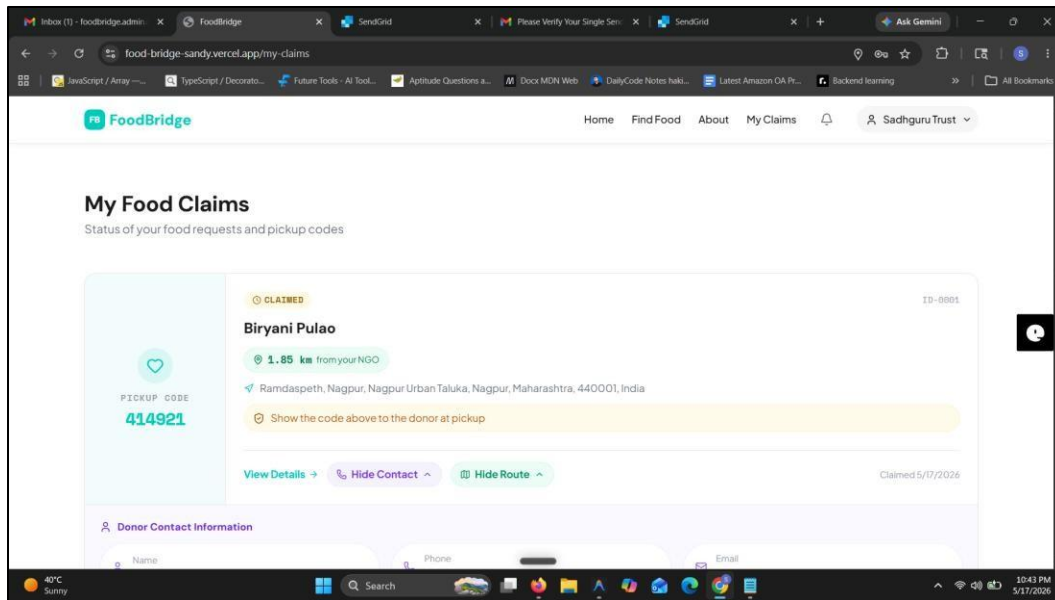


Fig. 5.11: My Food Claims — CLAIMED Status with 6-Digit Pickup OTP Code Display

Fig. 5.11 shows the expanded My Food Claims view after the NGO clicks 'Show Contact', revealing the Donor Contact Information panel. This panel displays the donor's full name (Sanket Prajapati), phone number (+91 8421927896), and email address (), enabling the NGO to coordinate pickup logistics with the donor directly.prajapatisanketssp321@gmail.com

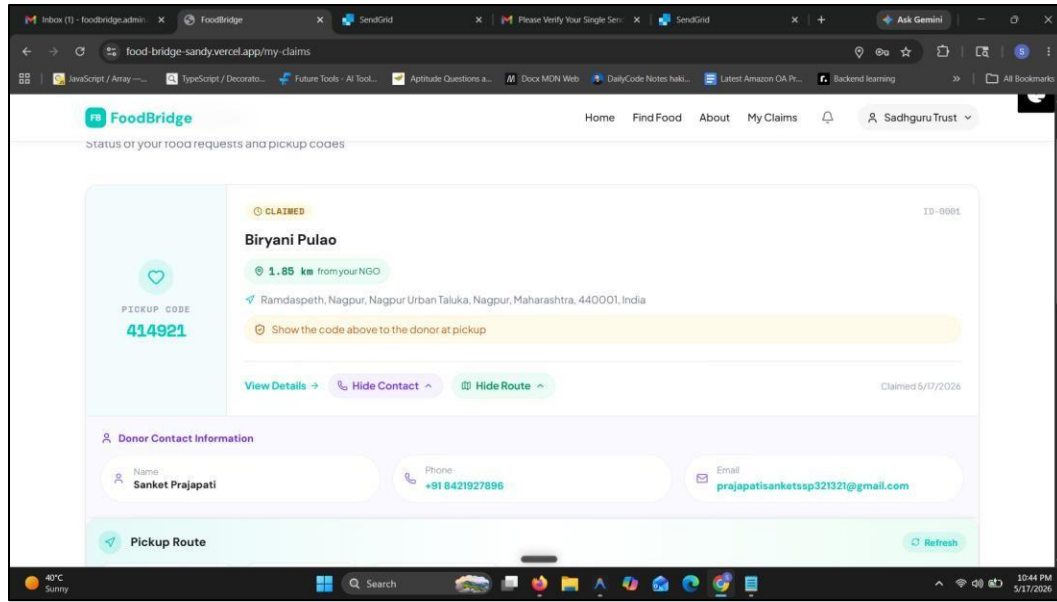


Fig. 5.12: My Food Claims — Donor Contact Information Revealed After Claim

Fig. 5.12 presents the Leaflet-powered interactive pickup route map, which is displayed when the NGO expands the 'Show Route' section of a claimed donation. The map renders OpenStreetMap tiles and plots two markers: a green marker at the NGO's registered location (Ramdaspath, Nagpur) and a red marker at the donation's pickup location. A highlighted route line is drawn between the two points to assist the NGO in navigating to the food pickup location. The map supports Street, Detailed, Satellite, and Topographic tile layer modes.

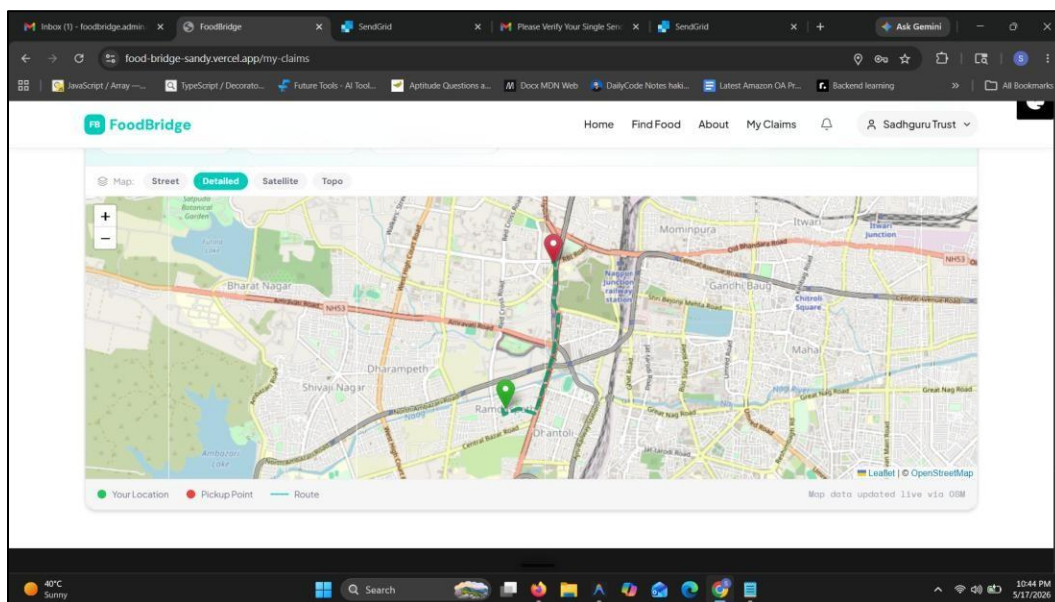


Fig. 5.13: Interactive Pickup Route Map — Leaflet with OpenStreetMap Tiles Showing Donor-to-NGO Route

Fig. 5.14 shows the My Food Claims page after successful OTP verification and pickup completion. The donation's status badge has been updated from CLAIMED to PICKED UP, confirming the successful completion of the food rescue transaction. The pickup code remains visible on the card as an audit reference, and the claim date is recorded as 5/17/2026.

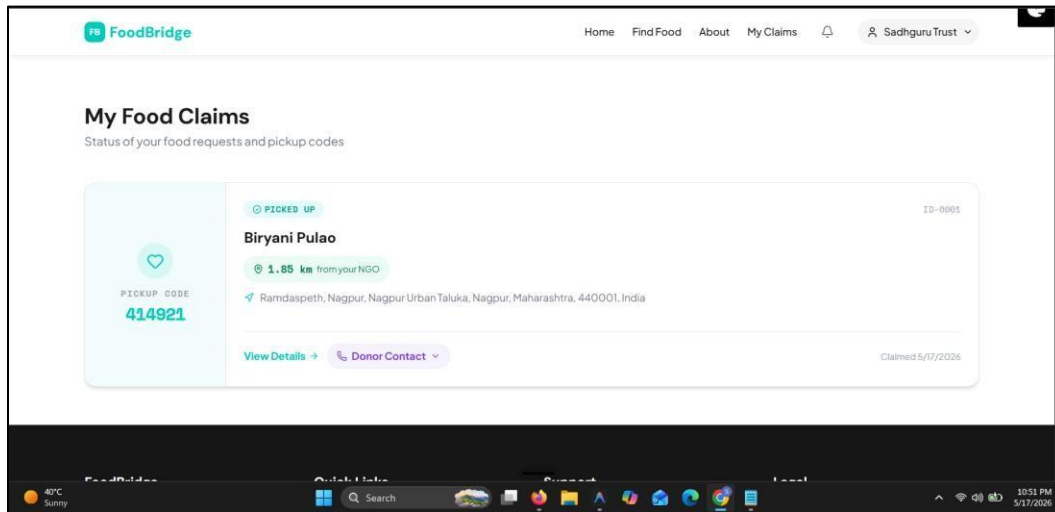


Fig. 5.14: My Food Claims — PICKED UP Confirmation After Successful OTP Verification

5.6.3 Notification System

The FoodBridge notification system provides both in-application notifications and email notifications for all key events in the food rescue workflow. Figures 5.14, 5.15, and 5.16 illustrate the in-application Notifications page as experienced by NGO and Donor users respectively.

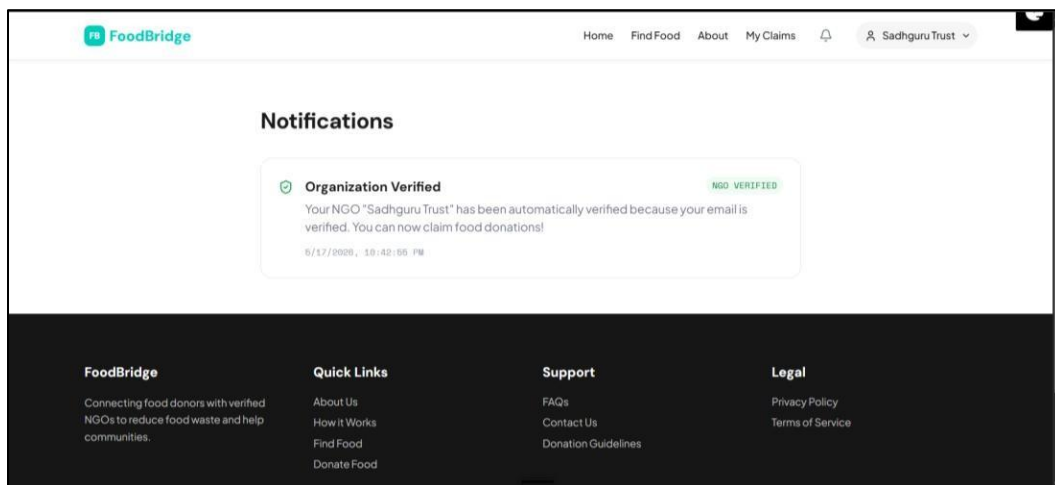


Fig. 5.15: NGO Notifications — Organisation Verification Confirmation Alert

Fig. 5.14 shows the NGO user's notification confirming that their organisation 'Sadhguru Trust' has been automatically verified because their email is verified, and they are now eligible to claim food donations. This notification was generated at 10:42:55 PM on 5/17/2026, immediately after the NGO's email verification was completed.

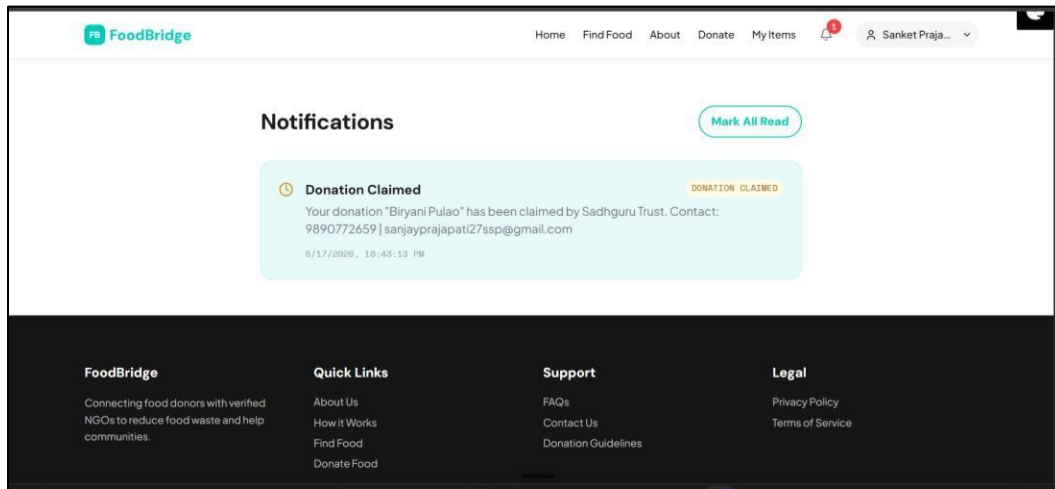


Fig. 5.16: Donor Notifications — Donation Claimed Notification with NGO Contact Details

Fig. 5.15 shows the donor's DONATION CLAIMED notification, which was generated when Sadhguru Trust claimed the 'Biryani Pulao' donation. The notification includes the NGO's contact phone number (9890772659) and email address for the donor's reference, enabling immediate coordination.

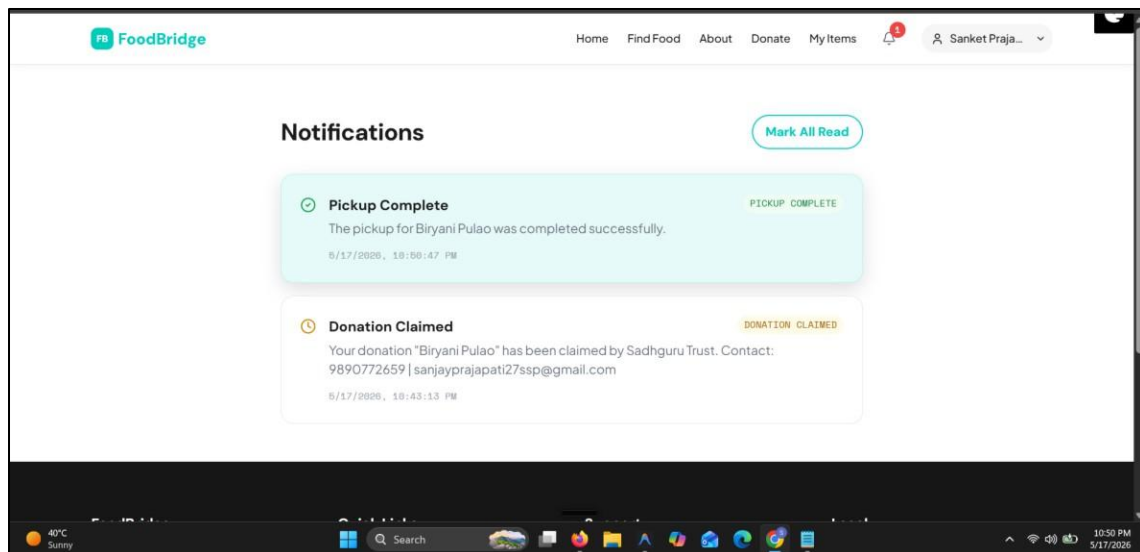


Fig. 5.17: Donor Notifications — Pickup Complete Confirmation and Donation Claimed History

Fig. 5.16 shows the donor's complete notification history following a successful food rescue transaction: a PICKUP COMPLETE notification confirming that the pickup for 'Biryani Pulao' was completed successfully at 10:50:47 PM, and the earlier DONATION CLAIMED notification. The notification centre provides a 'Mark All Read' button and displays notification timestamps with contextual type badges.

5.6.4 Email Notification System

The FoodBridge email notification system delivers HTML-formatted transactional emails for four distinct event types. All emails are rendered with the FoodBridge brand header (green background, white text), dark-themed body panels, and clear call-to-action elements. Figures 5.17 through 5.20 illustrate the email notifications as received by users on mobile devices.

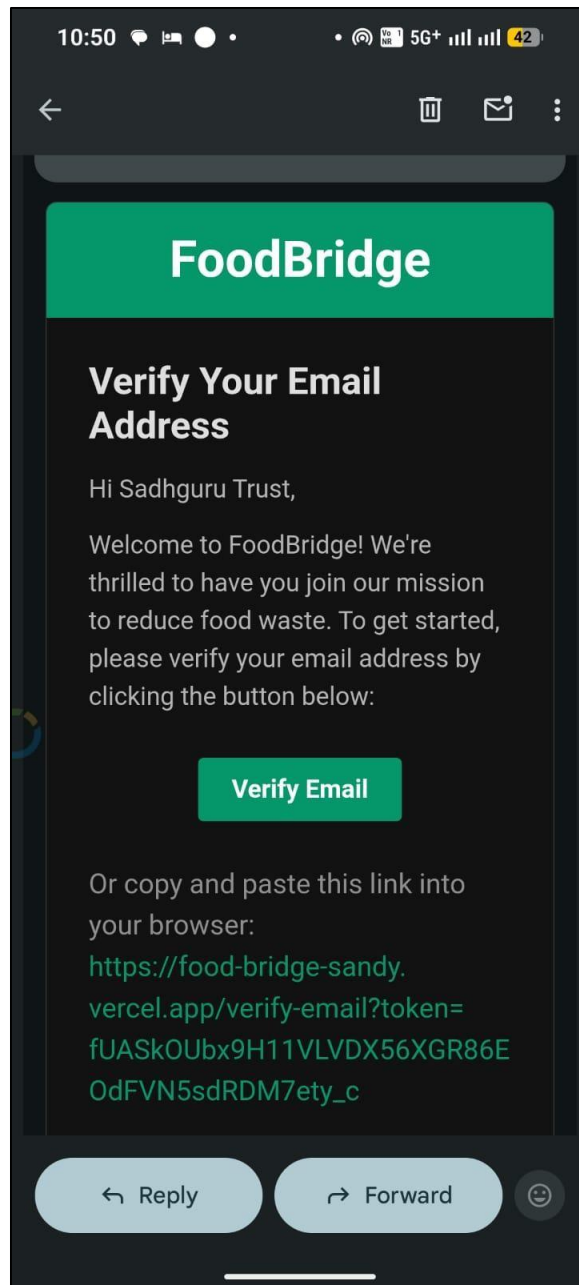


Fig. 5.18: Email — Verify Your Email Address (Registration Verification Flow)

Fig. 5.17 illustrates the email verification message sent to newly registered users. The email greets the user by name ('Hi Sadhguru Trust,'), explains the email verification requirement, and provides a prominent green 'Verify Email' call-to-action button. As a fallback, a complete verification URL is also included in plain text for copying into the browser.

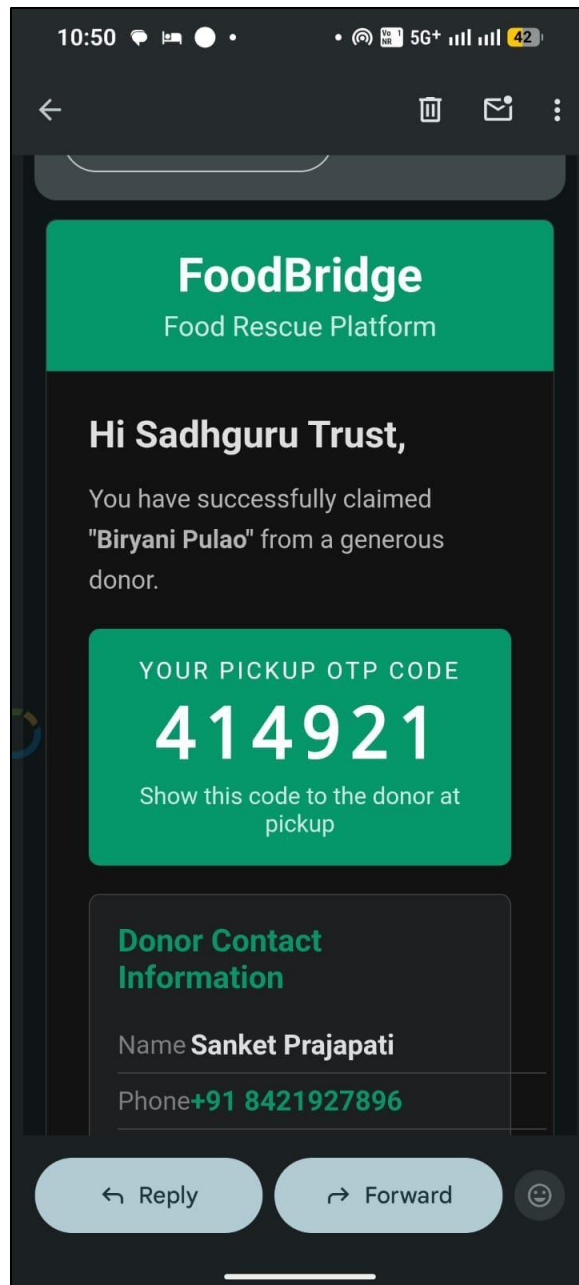


Fig. 5.19: Email — NGO Receives Pickup OTP Code and Donor Contact Information

Fig. 5.18 shows the email dispatched to the NGO (Sadhguru Trust) when a food claim is successfully created. The email confirms that the NGO has claimed 'Biryani Pulao' from a generous donor, presents the six-digit Pickup OTP Code (414921) in a prominent green panel with the instruction 'Show this code to the donor at pickup', and provides the Donor Contact Information panel displaying the donor's name (Sanket Prajapati) and phone number.

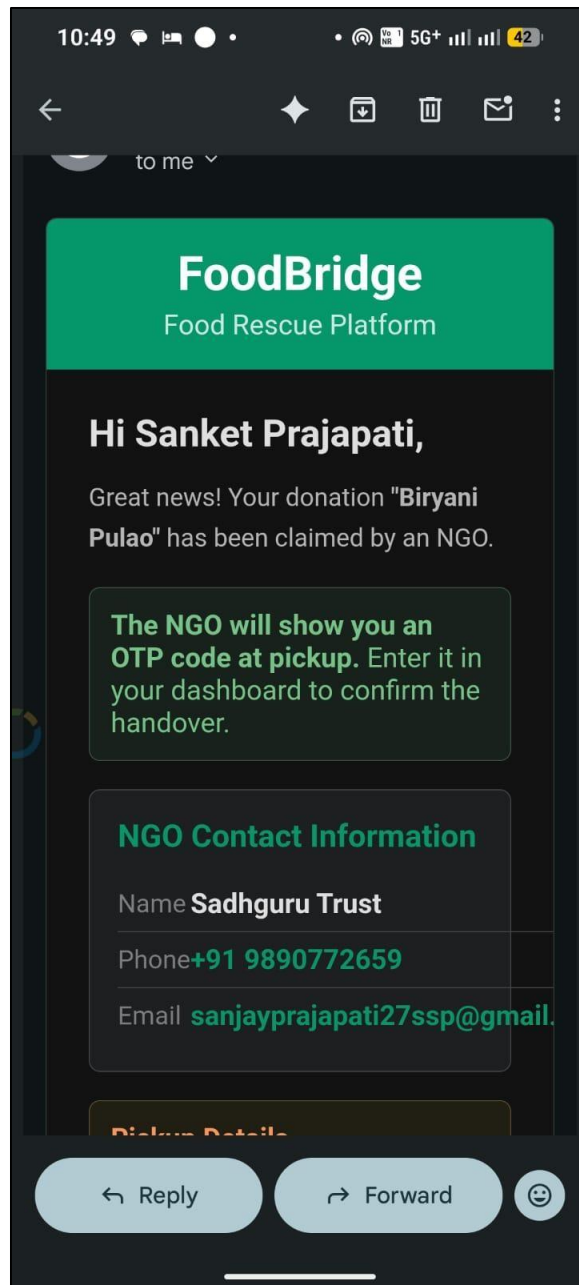


Fig. 5.20: Email — Donor Notified of Successful Claim with NGO Contact and OTP Instructions

Fig. 5.19 presents the email sent to the donor (Sanket Prajapati) when their donation has been claimed by an NGO. The email confirms that 'Biryani Pulao' has been claimed by an NGO, instructs the donor to await the NGO's OTP code at pickup and enter it in their dashboard to confirm the handover, and provides the NGO Contact Information panel with the NGO's name (Sadhguru Trust), phone number, and email address.

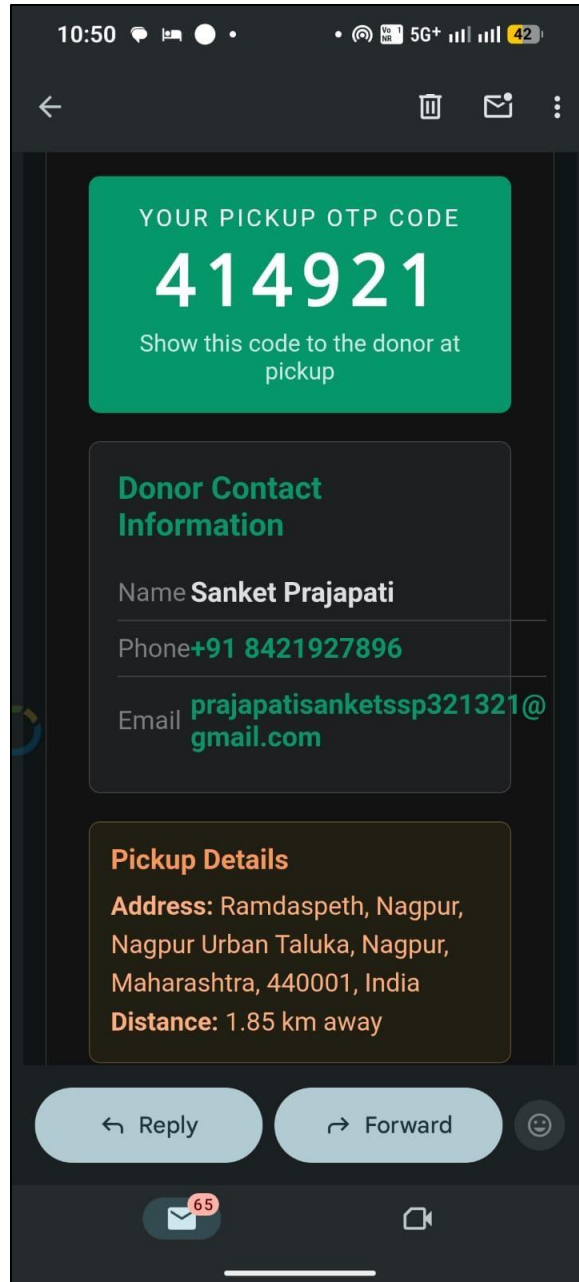


Fig. 5.20: Email — Donor Receives Full OTP Email Including Pickup Location and Distance

Fig. 5.20 shows the complete OTP email received by the donor, including the OTP code section, the Donor Contact Information panel (confirming Sanket Prajapati's contact details), and the Pickup Details panel specifying the pickup address (Ramdas Peth, Nagpur, Maharashtra, 440001) and the calculated distance (1.85 km away). This email provides the donor with all the information needed to prepare for and verify the food handover.

CHAPTER 6

CONCLUSIONS AND SUMMARY

6.1 Conclusions and Inferences

The FoodBridge project, developed over the course of a six-month internship at Kiran Academy, Pune, represents a complete, fully functional and deployable web application that successfully addresses the problem of food waste through digital technology. The application demonstrates that a carefully designed, technology-mediated platform can significantly reduce the friction involved in connecting food donors with recipient NGOs, thereby enabling a more efficient and accountable food rescue process.

The primary conclusion derived from the development and deployment of FoodBridge is that the combination of role-based access control, geospatial distance filtering, OTP-based pickup verification, and automated email notifications constitutes an effective and practically deployable solution framework for digital food rescue platforms in the Indian urban context. The OTP verification mechanism, in particular, represents a novel adaptation of a widely-understood security pattern to the food rescue domain, providing a lightweight yet effective means of confirming physical food handover without requiring complex hardware infrastructure.

The Haversine-based geospatial filtering mechanism demonstrated that accurate distance computation between geographic coordinates can be efficiently performed in Python without the need for external geospatial databases or GIS software, making the system cost-effective and straightforward to deploy. The geocoding integration with Geopy Nominatim, augmented by a hierarchical fallback strategy (from exact address to city level to state level), proved reliable in handling the varied address formats encountered in Indian urban addresses.

The decoupled architecture of FoodBridge — Django REST API backend and React SPA frontend — proved to be a sound design choice. The clear separation of concerns enabled parallel development of the two tiers and simplified testing, as each tier could be validated independently. The JWT-based stateless authentication system eliminated the need for server-side session management, simplifying horizontal scaling of the backend.

The production deployment of FoodBridge on free-tier cloud services — Render for the backend, Vercel for the frontend, Railway for the database, and Cloudinary for media storage — confirmed that a fully functional, feature-rich web application can be deployed and operated at effectively zero infrastructure cost, making such platforms viable for NGOs and social enterprises with limited budgets. The only significant operational constraint encountered was the SMTP connection timeout behaviour of the Render platform, which was resolved by implementing email dispatch in a background thread.

Testing of the end-to-end workflow confirmed that the complete sequence of Donor registration → Email verification → Donation posting → NGO registration → NGO verification → Donation claiming → Contact exchange → OTP verification → Pickup completion → Notification delivery operates correctly and reliably in the production environment. All email notifications were delivered successfully to the test user accounts, and all in-application notifications were generated and displayed correctly.

6.2 Lessons Learnt

This internship provided an exceptionally rich learning experience spanning the full breadth of modern web application development. The most significant technical lesson learnt was the importance of designing the data model and API contract before commencing implementation. The time invested in the database schema design and API endpoint specification at the beginning of the project paid substantial dividends throughout the development process, as it prevented the need for disruptive schema migrations and API refactoring in later stages.

The implementation of the background thread email sending mechanism to prevent SMTP timeouts from blocking HTTP request handlers provided a practical lesson in the importance of understanding the operational constraints of cloud hosting environments. This experience highlighted the need to design systems that gracefully handle the latency and connectivity characteristics of shared cloud infrastructure.

Working with geospatial data — geocoding addresses, storing latitude and longitude coordinates, and computing distances using the Haversine formula — provided valuable experience in a domain that is increasingly important in location-aware web applications.

The debugging of geocoding failures for non-standard Indian address formats led to the development of the hierarchical fallback geocoding strategy, which is a reusable pattern applicable to any location-aware application operating in the Indian context.

The experience of managing multiple cloud service integrations — Cloudinary, Railway PostgreSQL, Render, Vercel, Gmail SMTP, and SendGrid — within a single application reinforced the importance of environment variable management and the separation of development and production configurations. The use of python-dotenv for environment variable loading and the maintenance of separate .env and .env.production files for the frontend and backend respectively proved to be an effective configuration management strategy.

Finally, the process of preparing this systematic internship report provided valuable experience in technical writing and the structured documentation of complex software systems. The discipline of translating technical implementation details into clear, formal academic prose — supported by appropriately labelled figures, tables, and indexed references — is a professional skill of enduring value that will be applied throughout the subsequent stages of the student's professional career.

REFERENCES

- [1] Django Software Foundation. (2024). Django Documentation, Version 5.0. Retrieved from <https://docs.djangoproject.com/en/5.0/> (Accessed: 12 Jan. 2026).
- [2] Django REST Framework. (2024). Django REST Framework Documentation. Retrieved from <https://www.django-rest-framework.org/> (Accessed: 15 Jan. 2026).
- [3] Auth0. (2024). Introduction to JSON Web Tokens. Retrieved from <https://jwt.io/introduction/> (Accessed: 20 Jan. 2026).
- [4] Jada, Y. (2023). djangoestframework-simplejwt Documentation. Retrieved from <https://django-rest-framework-simplejwt.readthedocs.io/> (Accessed: 22 Jan. 2026).
- [5] Meta Platforms, Inc. (2024). React Documentation. Retrieved from <https://react.dev/> (Accessed: 05 Feb. 2026).
- [6] Vite. (2024). Vite Official Documentation, Version 5. Retrieved from <https://vitejs.dev/> (Accessed: 07 Feb. 2026).
- [7] Remix Software. (2024). React Router v7 Documentation. Retrieved from <https://reactrouter.com/> (Accessed: 10 Feb. 2026).
- [8] Tailwind Labs. (2024). Tailwind CSS Documentation, Version 3. Retrieved from <https://tailwindcss.com/docs/> (Accessed: 14 Feb. 2026).
- [9] Cloudinary. (2024). Cloudinary Documentation — Python SDK. Retrieved from https://cloudinary.com/documentation/django_integration (Accessed: 18 Feb. 2026).
- [10] Geopy Contributors. (2024). Geopy Documentation — Nominatim Geocoder. Retrieved from <https://geopy.readthedocs.io/> (Accessed: 22 Feb. 2026).
- [11] Leaflet. (2024). Leaflet.js — Interactive Maps for Browsers. Retrieved from <https://leafletjs.com/> (Accessed: 01 Mar. 2026).
- [12] OpenStreetMap. Retrieved from <https://www.openstreetmap.org/> (Accessed: 05 Mar. 2026).

- [13] WhiteNoise Authors. (2024). WhiteNoise Documentation. Retrieved from <https://whitenoise.readthedocs.io/> (Accessed: 10 Mar. 2026).
- [14] Gunicorn. (2024). Gunicorn — Python WSGI HTTP Server for UNIX. Retrieved from <https://gunicorn.org/> (Accessed: 15 Mar. 2026).
- [15] Railway Technologies Inc. (2024). Railway Documentation. Retrieved from <https://docs.railway.app/> (Accessed: 20 Mar. 2026).
- [16] Vercel Inc. (2024). Vercel Documentation. Retrieved from <https://vercel.com/docs/> (Accessed: 25 Mar. 2026).
- [17] Food and Agriculture Organization of the United Nations. (2019). The State of Food and Agriculture 2019: Moving Forward on Food Loss and Waste Reduction. FAO, Rome. ISBN 978-92-5-131789-1.
- [18] UNEP. (2021). UNEP Food Waste Index Report 2021. United Nations Environment Programme, Nairobi.
- [19] Papargyropoulou, E., Lozano, R., Steinberger, J. K., Wright, N., & bin Ujang, Z. (2014). The food waste hierarchy as a framework for the management of food surplus and food waste. *Journal of Cleaner Production*, 76, 106–115.
- [20] Schanes, K., Dobernig, K., & Gözet, B. (2018). Food waste matters — A systematic review of household food waste practices and their policy implications. *Journal of Cleaner Production*, 182, 978–991.
- [21] Sinha, A. (2022). dj-database-url — Database URL Configuration for Django. Retrieved from <https://pypi.org/project/dj-database-url/> (Accessed: 05 May 2026).

ANNEXURE A — DEPLOYMENT CONFIGURATION

This annexure documents the key deployment configuration files used to deploy the FoodBridge application to the production cloud environment.

A.1 Backend Procfile (Render / Railway)

```
web: gunicorn config.wsgi:application --bind 0.0.0.0:$PORT --workers 2 --
timeout 120
release: python manage.py migrate --noinput && python manage.py
collectstatic --noinput
```

A.2 Frontend vercel.json (SPA Rewrite Rule)

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

A.3 Key Backend Environment Variables

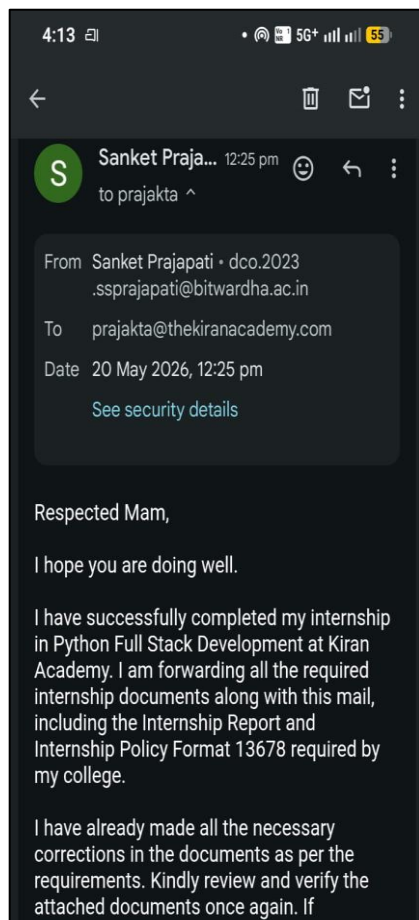
Table A.1: Backend Production Environment Variables

Variable	Description	Example Value
SECRET_KEY	Django secret key	django-insecure-...
DEBUG	Debug mode flag	False (production)
ALLOWED_HOSTS	Comma-separated allowed hostnames	your-app.onrender.com
DATABASE_URL	PostgreSQL connection URL	postgresql://...
CORS_ALLOWED_ORIGINS	Frontend URL for CORS	https://your-app.vercel.app
EMAIL_HOST_USER	Gmail SMTP username	youremail@gmail.com
EMAIL_HOST_PASSWORD	Gmail App Password	xxxx xxxx xxxx xxxx
FRONTEND_URL	Frontend base URL for email links	https://your-app.vercel.app
CLOUDINARY_CLOUD_NAME	Cloudinary cloud name	your_cloud_name
CLOUDINARY_API_KEY	Cloudinary API key	123456789012345
CLOUDINARY_API_SECRET	Cloudinary API secret	your_api_secret

ANNEXURE B — INDUSTRY MENTOR DOCUMENT VERIFICATION

This annexure presents the email correspondence confirming that all internship-related documents — including the internship report, internship policy documents, and course completion certificate — were reviewed and digitally verified by Ms. Prajakta V. Murmure, Industry Mentor, Kiran Academy, Pune. The screenshots below illustrate the email thread received as proof of verification.

Snapshot B.1: Email Verification



Snapshot B.2: Email Verification

